

A Project report on

**REAL TIME FIRE AND SMOKE DETECTION SYSTEM
USING YOLOV9 WITH INTELLIGENT ALERTING**

Submitted in partial fulfillment of the requirements

for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

By

D. NEHA ANJUM (224G1A0559)

V. MEGHANA (234G5A0507)

S. MOHAMMAD ANEES (224G1A0553)

Under the Guidance of

Dr D. Rajesh Babu, MTech, PhD

Associate Professor



Department of Computer Science & Engineering

**SRINIVASA RAMANUJAN INSTITUTE OF TECHNOLOGY
(AUTONOMOUS)**

Rotarypuram Village, B K Samudram Mandal, Ananthapuramu - 515701

2025-2026

SRINIVASA RAMANUJAN INSTITUTE OF TECHNOLOGY

(AUTONOMOUS)

(Affiliated to JNTUA, Accredited by NAAC with 'A' Grade, Approved by AICTE, New Delhi & Accredited by NBA (EEE, ECE & CSE))

Rotarypuram Village, BK Samudram Mandal, Ananthapuramu-515701

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



Certificate

This is to certify that the project report entitled **Real Time Fire and Smoke Detection System Using YOLOV9 with Intelligent Alerting** is the bonafide work carried out by **D. Neha Anjum, V. Meghana, S. Mohammad Anees** bearing Roll Number **224G1A0559, 234G5A0507, 224G1A0553** in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology** in **Computer Science & Engineering** during the academic year 2025-2026.

Project Guide

Dr D. Rajesh Babu, MTech, PhD
Associate Professor

Head of the Department

Dr P. Veera Prakash, MTech, PhD
Assistant Professor & HOD

Date:

Place: Rotarypuram

External Examiner

DECLARATION CERTIFICATE

We students of **Computer Science & Engineering, SRINIVASA RAMANUJAN INSTITUTE OF TECHNOLOGY(AUTONOMOUS)**, Rotarypuram, hereby declare that the dissertation entitled **Real Time Fire and Smoke Detection System Using YOLOV9 with Intelligent Alerting** embodies the report of our project work carried out by us during IV year under the guidance of Dr. D. Rajesh Babu, MTech, PhD, Associate Professor, Department of Computer Science & Engineering, Srinivasa Ramanujan Institute of Technology, and this work has been submitted for the partial fulfillment of the requirements for the award of degree of Bachelor of Technology.

The results embodied in this project report have not been submitted to any other University or Institute for the award of any Degree or Diploma.

Date:

Place:

S.No.	Name of the Student	Roll Number	Signature
1	D. Neha Anjum	224G1A0559	
2	V. Meghana	234G5A0507	
3	S. Mohammad Anees	224G1A0553	

Vision & Mission of the SRIT

Vision:

To become a premier Educational Institution in India offering the best teaching and learning environment for our students that will enable them to become complete individuals with professional competency, human touch, ethical values, service motto, and a strong sense of responsibility towards environment and society at large.

Mission:

- Continually enhance the quality of physical infrastructure and human resources to evolve in to a center of excellence in engineering education.
- Provide comprehensive learning experiences that are conducive for the students to acquire professional competences, ethical values, life-long learning abilities and understanding of the technology, environment and society.
- Strengthen industry institute interactions to enable the students work on realistic problems and acquire the ability to face the ever changing requirements of the industry.
- Continually enhance the quality of the relationship between students and faculty which is a key to the development of an exciting and rewarding learning environment in the college.

Vision & Mission of the Department of CSE

Vision:

To evolve as a leading department by offering best comprehensive teaching and learning practices for students to be self-competent technocrats with professional ethics and social responsibilities.

Mission:

DM 1: Continuous enhancement of the teaching-learning practices to gain profound knowledge in theoretical & practical aspects of computer science applications.

DM 2: Administer training on emerging technologies and motivate the students to inculcate self-learning abilities, ethical values and social consciousness to become competent professionals.

DM 3: Perpetual elevation of Industry-Institute interactions to facilitate the students to work on real-time problems to serve the needs of the society.

Program Educational Objectives (PEOs)

An SRIT graduate in Computer Science & Engineering, after three to four years of graduation will:

PEO 1: Lead a successful professional career in IT / ITES industry / Government organizations with ethical values.

PEO 2: Become competent and responsible computer science professional with good communication skills and leadership qualities to respond and contribute significantly for the benefit of society at large.

PEO 3: Engage in life-long learning, acquiring new and relevant professional competencies / higher academic qualifications.

ACKNOWLEDGEMENT

The successful completion of this project would be incomplete without acknowledging those whose constant support and guidance made it possible. We are grateful for the opportunity to express our thanks to all of them.

We express our sincere gratitude to our guide, **Dr D. Rajesh Babu, Associate Professor, Computer Science & Engineering**, for his continuous guidance, encouragement, and constructive feedback at every stage of the project, which greatly contributed to its successful completion.

We express our deep felt gratitude to **Mr. K. Lokeshnath, Assistant Professor and Mr. M. Narasimhulu, Assistant Professor, Project Coordinators** for their valuable guidance and unstinting encouragement enabled us to accomplish our project successfully in time.

We are very much thankful to **Dr P. Veera Prakash, Assistant Professor & Head of the Department, Computer Science & Engineering**, for his kind support and for providing necessary facilities to carry out the work.

We extend our special thanks to **Dr G. Balakrishna, Principal of Srinivasa Ramanujan Institute of Technology**, for providing the necessary information for our project. We also thank the faculty, non-teaching staff, and friends who directly or indirectly supported us in completing it on time.

We sincerely thank the Management for the excellent facilities and our families for their constant support throughout the project.

Project Associates

Neha Anjum D (224G1A0559)

Meghana V (234G5A0507)

Mohammad Anees S (224G1A0553)

ABSTRACT

Fire accidents pose a serious threat to human life, property, and environmental safety, especially in residential areas, industries, forests, and agricultural regions. Traditional fire detection systems such as smoke detectors and heat sensors often suffer from delayed responses, limited coverage, and high false alarm rates. To address these challenges, this study proposes an intelligent real-time fire detection system based on deep learning and computer vision techniques. The proposed system utilizes the YOLOv9 object detection model to accurately identify fire and smoke from surveillance video streams. The system continuously captures video frames from cameras and processes them using a trained deep learning model capable of detecting fire-related visual patterns with high precision. A temporal validation mechanism is incorporated to verify detections across consecutive frames, which helps reduce false alarms caused by lighting variations, reflections, or environmental disturbances. In addition, the system evaluates the severity of detected fire incidents by analyzing parameters such as bounding box size, detection confidence, and smoke density, categorizing them into different risk levels. An automated alert mechanism is integrated to notify monitoring personnel immediately when a fire event is confirmed. The system also maintains detection logs and provides a monitoring interface for real-time visualization and analysis of fire incidents.

Keywords: *Fire detection, YOLOv9, Deep learning, Real-time surveillance, Smoke detection and Emergency alert system.*

PROJECT CONTRIBUTION TOWARDS SUSTAINABLE DEVELOPMENT GOALS

Project title:

Real-Time Fire and Smoke Detector using YOLOV9 with Intelligent Alerting.

SDG – Project Mapping:

S.No.	Sustainable Development Goals	Mapped (Yes / No)
1	SDG 1: No Poverty	No
2	SDG 2: Zero Hunger	No
3	SDG 3: Good Health and Well-Being	Yes
4	SDG 4: Quality Education	No
5	SDG 5: Gender Equality	No
6	SDG 6: Clean Water and Sanitation	No
7	SDG 7: Affordable and Clean Energy	No
8	SDG 8: Decent Work and Economic Growth	Yes
9	SDG 9: Industry Innovation and Infrastructure	Yes
10	SDG 10: Reduced Inequalities	No
11	SDG 11: Sustainable Cities and Communities	Yes
12	SDG 12: Responsible Consumption and Production	Yes
13	SDG 13: Climate Action	Yes
14	SDG 14: Life Below Water	No
15	SDG 15: Life on Land	Yes
16	SDG 16: Peace Justice and Strong Institutions	Yes
17	SDG 17: Partnerships for the Goals	Yes

Explanation:

- The project helps in **early** detection of **fire and smoke**. Reducing risks to **human life** and improve safety.
- Uses AI (YOLOv9) for accurate and real-time monitoring, minimizing human dependency.
- Provides **instant intelligent alerts**, helping authorities take quick action.
- Supports **smart city safety systems** and disaster management.
- Helps in **environmental protection** by detecting fire hazards early and reducing damage.
- Encourages adoption of AI-based automation in surveillance system

Expected Impact:

Parameter	Impact
Detection Speed	Real-time detection within seconds
Accuracy Improvement	High accuracy using YOLOv9 model
Safety Enhancement	Early warning reduces damage and saves lives
Manual Monitoring	Reduces need for continuous human surveillance

CONTENTS

	Page No
List of Figures	vi
Abbreviations	vii
Chapter 1	Introduction
	1 - 9
1.1	Problem Statement
	2
1.2	Objectives
	2
1.3	Scope of the Project
	3
1.4	OpenCV
	3
1.4.1	Basic Terminology
	4
1.4.2	Types of Deep Learning
	5
1.4.3	Deep Learning Approaches in Project
	6
1.5	Models
	7
15.1	YOLOv9
	7
1.6	Organization of Thesis
	9
Chapter 2	Literature Survey
	10 - 16
Chapter 3	Planning
	17 - 27
3.1	Existing System
	17
3.1.1	Limitations of the Existing System
	17
3.2	Proposed System
	18
3.2.1	Core Components of the Proposed System
	19
3.2.2	Advantages of the Proposed System
	20
3.3	Software Development Methodology
	21
3.3.1	Iterative Methodology Overview
	22
3.3.2	Why Iterative Works for This Project
	24
3.3.3	Supports Continuous Model Performance
	24
3.3.4	Flexibility in Model Selection
	25
3.3.5	Early Error Detection
	25
3.3.6	Efficient Data Handling
	26
3.3.7	Improved System Performance
	26
3.4	Chapter Summary
	27
Chapter 4	System Requirements Specification
	28 - 41
4.1	Functional Requirements
	29
4.1.1	Basic Requirements
	29
4.1.2	Dataset Description
	30
4.1.3	Data Collection
	30
4.1.4	Data Features
	31
4.1.5	Data Augmentation
	31

4.2 Non-Functional Requirements	34
4.3 Hardware Requirements	35
4.4 Software Requirements	37
4.5 Python Libraries	39
4.6 Chapter Summary	41
Chapter 5 System Analysis and Design	42 - 51
5.1 UML Diagrams	42
5.1.1 Use Case Diagram	43
5.1.2 Class Diagram	44
5.1.3 Sequence Diagram	45
5.1.4 Activity Diagram	46
5.1.5 Component Diagram	47
5.2 System Architecture	48
5.3 System Flowchart	49
5.4 Chapter Summary	51
Chapter 6 Implementation	52 - 63
6.1 Development Environment	52
6.2 Dataset Preparation	54
6.3 Image Preprocessing	55
6.4 Data splitting	57
6.5 Model Implementation	58
6.6 Model Compilation and Training	59
6.7 Model Evaluation	60
6.8 Best Model Selection	61
6.9 Streamlit Application	62
6.10 System Workflow	63
6.11 Chapter Summary	63
Chapter 7 Testing	64 - 66
Chapter 8 Results	67 - 70
Conclusion and Future Work	71
References	72
Publications	73 - 82
Internship Certificates	83 - 85
Project CO-PO Mapping	86
Details of Project Team	87

LIST OF FIGURES

Fig. No.	Description
1.1	Types of Deep Learning
1.2	YOLOv9 Architecture
3.1	Iterative Software Development model
4.1	System Requirement Specifications
4.2	Confusion matrix classification outcomes
5.1	Use Case Diagram
5.1	Class Diagram
5.3	Sequence Diagram
5.4	Activity Diagram
5.5	Component Diagram
5.6	System Architecture
5.7	Flowchart of the System
6.1	Import Libraries
6.2	Dataset Collection
6.3	Preparing Dataset Used for YOLOv9 Model Training
6.4	Data Preprocessing
6.5	Data Splitting
6.6	Model Implementation
6.7	Model Training
6.8	Model Testing
6.9	Home Page
8.1	Image Based Detections
8.2	Video Based Predictions
8.3	Live Analysis
8.4	Logs Page

LIST OF ABBREVIATIONS

Abbreviation	Full Form
YOLOv9	You Only Look Once version 9
CNN	Convolutional Neural Network
IoT	Internet of Things
FPS	Frames Per Second
GPU	Graphics Processing Unit
API	Application Programming Interface
UI	User Interface
SVM	Support Vector Machine
RGB	Red Green Blue (Color Channels)
ML	Machine Learning

CHAPTER 1

INTRODUCTION

Fire accidents are one of the most dangerous disasters that can cause severe damage to human life, infrastructure, and the environment. Rapid urbanization, industrial growth, and increased use of electrical and combustible materials have significantly increased the risk of fire incidents in residential, commercial, and forest areas. Traditional fire detection systems such as smoke detectors, heat sensors, and manual surveillance methods are widely used for fire monitoring. However, these conventional systems often suffer from several limitations, including delayed detection, limited monitoring coverage, and high false alarm rates caused by environmental conditions such as dust, fog, or lighting variations. As a result, there is a growing need for more intelligent and automated fire detection systems that can provide early warnings and improve emergency response times.

Recent advancements in artificial intelligence, deep learning, and computer vision have opened new possibilities for developing intelligent fire detection systems capable of analyzing visual data in real time. Deep learning-based object detection models, particularly the YOLO (You Only Look Once) family of algorithms, have demonstrated remarkable performance in detecting objects from images and video streams with high accuracy and speed. In this work, a real-time fire detection system is proposed using the YOLOv9 model to identify fire and smoke from surveillance camera feeds. The system continuously analyzes video frames, detects fire-related patterns, and validates detections across multiple frames to minimize false alarms.

1.1 Problem Statement

Fire accidents can occur unexpectedly in residential, industrial, agricultural, and forest environments, often leading to severe damage to life, property, and natural resources. Traditional fire detection systems such as smoke detectors and heat sensors are limited in their ability to provide early and reliable detection, as they typically respond only after smoke or heat reaches a certain threshold. These systems also have restricted coverage areas and may produce false alarms due to environmental factors such as dust, fog, or lighting variations. Manual monitoring through surveillance cameras is inefficient and requires continuous human attention.

1.2 Objectives

- i. To develop an intelligent fire detection system using deep learning techniques capable of identifying fire and smoke from surveillance video streams in real time.
- ii. To improve detection accuracy and reduce false alarms by implementing temporal validation and confidence-based filtering of fire detections.
- iii. To design a monitoring and alert system that provides real-time notifications and visualization through a user interface for faster emergency response and fire management.

1.3 Scope of the Project

The scope of this project focuses on developing an intelligent real-time fire detection system using deep learning and computer vision techniques. The system is designed to analyze video streams from surveillance cameras and automatically detect fire and smoke using the YOLOv9 object detection model. It aims to provide early detection of fire incidents in environments such as residential buildings, industrial areas, forests, and public spaces. The project also includes mechanisms for reducing false alarms through multi-frame validation and classifying fire severity levels for effective emergency response. Additionally, the system provides a monitoring interface for real-time visualization and alert generation.

1.4 OpenCV

OpenCV (Open Source Computer Vision Library) is a widely used open-source library designed for real-time computer vision and image processing applications. It provides a large collection of functions and algorithms for tasks such as image processing, object detection, video analysis, and feature extraction. In this project, OpenCV is used to capture video streams from surveillance cameras, process video frames, and perform operations such as frame extraction, image resizing, and visualization of detection results. It also helps in displaying bounding boxes around detected fire and smoke regions identified by the deep learning model.

1.4.1 Basic Terminology

Fire detection using deep learning and computer vision involves several fundamental concepts and terminologies that help in understanding how intelligent surveillance systems detect fire from images and videos. Some of the basic terms used in this domain are explained below:

- **Computer Vision:** Computer vision is a field of artificial intelligence that enables computers to interpret and understand visual information from images and videos. It allows systems to automatically detect objects, patterns, and events such as fire or smoke.
- **Deep Learning:** Deep learning is a subset of machine learning that uses artificial neural networks with multiple layers to learn complex patterns from large datasets. It is widely used for image recognition, object detection, and video analysis.
- **Object Detection:** Object detection is a computer vision technique used to identify and locate objects within an image or video frame. It not only classifies objects but also draws bounding boxes around detected objects such as fire or smoke.
- **Bounding Box:** A bounding box is a rectangular box drawn around an object detected in an image or video frame. It helps in visually identifying the location of detected fire or smoke.
- **Confidence Score:** A confidence score represents the probability that the detected object belongs to a particular class. Higher confidence scores indicate stronger certainty in the detection.

1.4.2 Types of Deep Learning Models

Deep learning models can be categorized into several types depending on their architecture and the problems they are designed to solve.

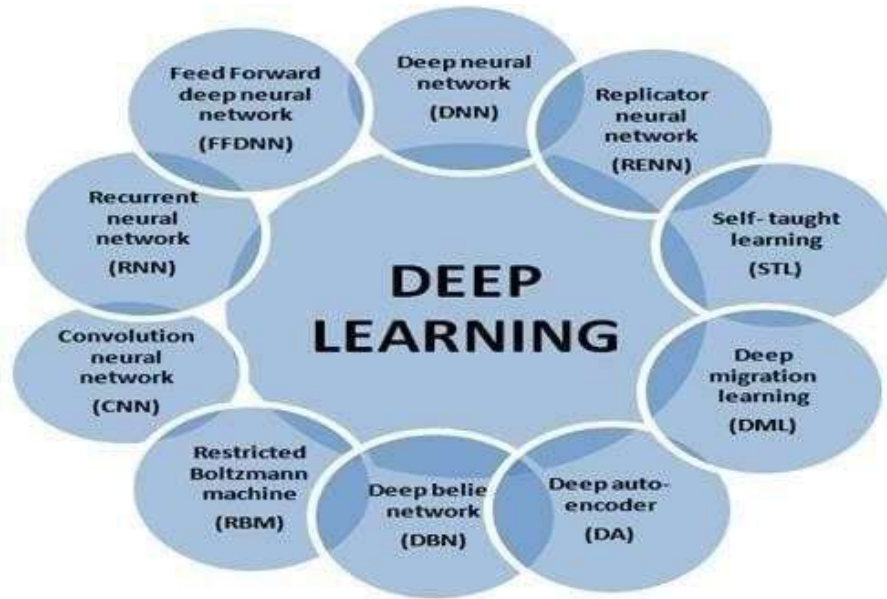


Fig.1.1: Types of Deep Learning

- **Convolutional Neural Networks (CNNs):** CNNs are deep learning models primarily used for analyzing images and videos. They automatically extract spatial features such as edges, shapes, and textures, making them suitable for fire and smoke detection tasks.
- **Recurrent Neural Networks (RNNs):** RNNs are designed to process sequential data where the output depends on previous information. They are commonly used in applications such as speech recognition, time-series prediction, and language processing.

- **Deep Neural Networks (DNNs):** DNNs consist of multiple fully connected layers that learn complex relationships between input and output data. They are used in classification, regression, and pattern recognition tasks.
- **Object Detection Networks:** Object detection networks combine classification and localization techniques to identify objects and determine their positions in images or video frames. Models such as YOLO, SSD, and Faster R-CNN are widely used for real-time detection tasks. Among these architectures, object detection models based on CNN architectures, particularly YOLO, are most suitable for real-time fire detection.

1.4.3 Deep Learning Approaches in project

In this project, a deep learning-based object detection approach is used to detect fire and smoke from surveillance video streams. Instead of manually designing features, the model automatically learns visual patterns such as flame shapes, smoke density, and color variations from the training dataset. The dataset used for training consists of labeled images containing fire and smoke objects. During preprocessing, images are resized, normalized, and divided into training, validation, and testing datasets. The training dataset is used to teach the model to recognize fire-related patterns, while the validation dataset helps tune model parameters and avoid overfitting. The testing dataset is used to evaluate the final model performance. The trained model analyzes each video frame and predicts whether fire or smoke is present. If detected, the system generates bounding boxes around the fire regions along with confidence scores.

1.5 Models

In this project, the YOLOv9 (You Only Look Once Version 9) object detection architecture is used for fire and smoke detection. YOLOv9 is selected because of its high detection accuracy, real-time processing capability, and ability to detect multiple objects within a single image. Unlike traditional detection methods that process images in multiple stages, YOLO performs detection in a single forward pass through the network, making it extremely fast and suitable for real-time surveillance systems. The YOLOv9 model learns spatial features of fire and smoke from images during training and later uses this knowledge to detect fire incidents in new video frames. The model predicts bounding boxes, class labels, and confidence scores simultaneously, allowing accurate localization of fire regions.

1.5.1. YOLOv9

YOLOv9 is an advanced deep learning-based object detection model designed for high-speed and high-accuracy detection tasks. It belongs to the YOLO family of algorithms that perform object detection using a single neural network architecture. The architecture of YOLOv9 consists of three major components: Backbone, Neck, and Head. The backbone network is responsible for extracting important visual features from the input images. It processes images through multiple convolutional layers to identify patterns such as edges, shapes, textures, and color variations related to fire and smoke. The neck network combines features from different layers of the backbone to improve detection performance.

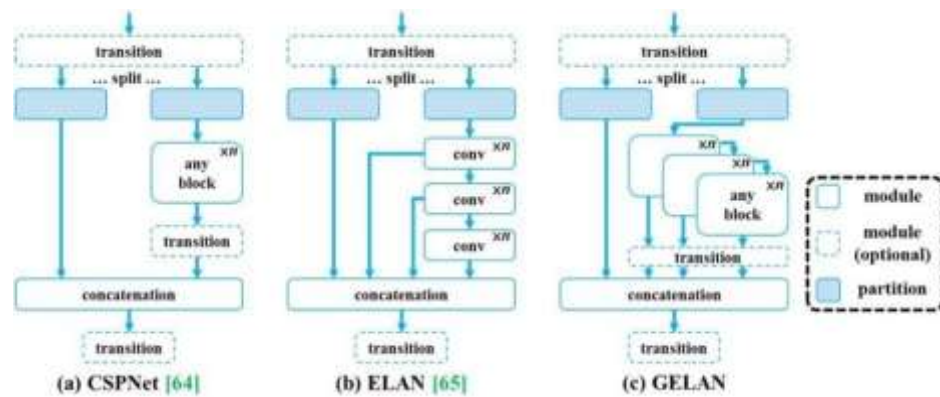


Fig.1.2: YOLOv9 Architecture

It helps the model detect objects at different scales, making it capable of identifying both small and large fire regions. The head network performs the final detection task by predicting bounding boxes, object classes, and confidence scores for each detected object. This enables the model to accurately locate fire and smoke in video frames. YOLOv9 also introduces improvements in training strategies and feature representation, which help increase detection accuracy while maintaining real-time processing speed. In this project, YOLOv9 is trained using a fire detection dataset and deployed for monitoring surveillance video streams.

1.6 Organization of the Thesis

This thesis is structured into multiple chapters, each covering a critical aspect of the research and development process:

- **Chapter 1:** Introduction – problem statement, objectives, scope, Deep Learning, Models and thesis structure.
- **Chapter 2:** Literature Survey – authors, summary and methodology.
- **Chapter 3:** Planning – Discusses limitations of existing systems, outlines the proposed approach and software development methodology.
- **Chapter 4:** System Requirement Specifications – Covers functional requirements, non-functional requirements, Python Libraries and hardware/software configurations.
- **Chapter 5:** System Analysis and Design – Explains the architectural design, UML diagrams.
- **Chapter 6:** Implementation – Details the image preprocessing, data splitting, Model compilation, training and web application.
- **Chapter 7:** Results and Discussion – Analyzes model performance, accuracy metrics.
- **Chapter 8:** Conclusion and future work.

CHAPTER 2

LITERATURE SURVEY

[1] Real-Time Fire Detection Using Deep Learning and Computer Vision

Authors: Y. Chen, H. Zhang, and L. Wang

Summary: This study presents a deep learning-based system for detecting fire in real-time using surveillance camera feeds. The authors emphasize that traditional fire detection systems often suffer from slow response times and limited monitoring capabilities. The proposed approach uses convolutional neural networks to detect fire patterns directly from images and video streams. Experimental results show that the deep learning model significantly improves fire detection accuracy and reduces false alarms compared to traditional sensor-based systems.

Methodology: The researchers used a CNN-based model trained on a large dataset of fire and smoke images. Video frames were extracted and processed through the neural network to detect fire regions. The performance was evaluated using accuracy, precision, and recall metrics.

[2] Forest Fire Detection Using Deep Learning Techniques

Authors: M. Sharma, P. Gupta, and R. Kumar

Summary: This research focuses on detecting forest fires using deep learning algorithms and remote camera monitoring systems. The system analyzes visual features such as flame color, smoke patterns, and motion characteristics to detect fire incidents at an early stage. The authors highlight that early fire detection can significantly reduce environmental damage and loss of wildlife.

Methodology: The study implemented convolutional neural networks trained on forest fire datasets. Image preprocessing techniques were applied to enhance detection accuracy. The model was evaluated using performance metrics including detection accuracy and false alarm rate.

[3] Fire Detection in Surveillance Systems Using YOLOv5

Authors: J. Li, X. Zhao, and Q. Huang

Summary: This paper proposes a real-time fire detection system based on the YOLOv5 object detection model. The system analyzes live surveillance video streams to identify fire and smoke in various environments. The results demonstrate that YOLO-based detection methods provide faster detection speeds and higher accuracy compared to traditional machine learning techniques.

Methodology: The authors trained the YOLOv5 model on a labeled fire detection dataset and implemented real-time detection using surveillance cameras. The model's performance was evaluated using precision, recall, and mean average precision (mAP).

[4] Early Fire Detection Using Artificial Intelligence and Image Processing

Authors: S. Patel and A. Desai

Summary: This study explores the use of artificial intelligence combined with image processing techniques for early fire detection. The authors emphasize that intelligent systems can identify fire at its early stages by analyzing color, motion, and smoke characteristics. The proposed system aims to reduce response time and improve fire safety monitoring.

Methodology: The system processes images using feature extraction methods and applies machine learning algorithms for classification. The model was trained using a

fire image dataset and evaluated using classification accuracy.

[5] Deep Learning-Based Fire and Smoke Detection in Video Surveillance

Authors: T. Nguyen, D. Pham, and H. Tran

Summary: This research presents a deep learning-based fire and smoke detection system designed for real-time video surveillance environments. The authors demonstrate that deep learning models can effectively identify fire and smoke patterns in complex scenes such as buildings and industrial areas.

Methodology: The proposed method uses a convolutional neural network trained on a fire detection dataset. The system extracts frames from video streams and processes them through the trained model to detect fire and smoke regions.

[6] AI-Based Intelligent Fire Monitoring System

Authors: K. Singh, R. Mehta, and S. Gupta

Summary: This paper introduces an intelligent fire monitoring system that uses artificial intelligence to detect fire incidents in indoor and outdoor environments. The system integrates computer vision with automated alert mechanisms to improve fire safety.

Methodology: The authors developed a deep learning model trained on fire image datasets and integrated it with surveillance cameras. The model performance was evaluated using accuracy and detection time metrics.

[7] Vision-Based Fire Detection System for Smart Cities

[8] Authors: L. Zhang and Y. Sun

Summary: This research focuses on implementing a vision based fire detection system

for smart city applications. The system aims to monitor public areas using surveillance cameras and automatically detect fire incidents in real time.

Methodology: The authors applied deep learning models for object detection and trained them using fire and smoke datasets. The system was tested in simulated urban environments to evaluate detection performance.

[9] Automatic Fire Detection Using Machine Learning Algorithms

Authors: R. Kumar and P. Mishra

Summary: This paper investigates the use of machine learning techniques for automatic fire detection. The study demonstrates that machine learning models can detect fire in images with improved accuracy compared to traditional rule-based approaches.

Methodology: The authors used feature extraction techniques and trained classification models such as SVM and Random Forest using fire image datasets.

[10] Fire and Smoke Detection Using Convolutional Neural

Networks Authors: A. Garcia and J. Lopez

Summary: This study proposes a CNN-based model for detecting fire and smoke in images and videos. The results show that deep learning models are capable of identifying fire patterns even under challenging conditions such as low lighting and background noise.

Methodology: The system processes image frames using convolutional layers to extract visual features and classify them as fire or non-fire.

[11] Smart Fire Detection System Using Deep Learning

Authors: H. Kim and S. Lee

Summary: This paper presents a smart fire detection system that combines deep learning algorithms with IoT devices for monitoring fire incidents. The system is designed to improve safety in residential and industrial environments. **Methodology:** The authors used a deep learning model trained on fire datasets and integrated it with IoT sensors for real-time monitoring.

[12] Fire Detection Using YOLO Object Detection Model

Authors: M. Ali and A. Khan

Summary: This research highlights the effectiveness of YOLO-based object detection models for identifying fire in real-time video streams. The authors demonstrate that YOLO models provide high detection speed and accuracy suitable for emergency monitoring systems.

Methodology: The YOLO model was trained on labeled fire datasets and evaluated using detection accuracy and processing speed metrics.

[13] Computer Vision-Based Fire Detection System

Authors: S. Johnson and M. Brown

Summary: The paper proposes a computer vision-based approach for detecting fire incidents using surveillance camera feeds. The system automatically identifies fire patterns and generates alerts when fire is detected. **Methodology:** Image processing techniques combined with machine learning algorithms were used to detect fire regions in video frames.

[13] Deep Neural Networks for Early Fire Detection

Authors: R. Verma and P. Singh

Summary: This study investigates the use of deep neural networks for detecting fire incidents in their early stages. The proposed model demonstrates improved detection accuracy and reduced false alarms compared to traditional systems.

Methodology: The authors trained a deep neural network using a dataset of fire images and evaluated its performance using accuracy and precision metrics.

[14] Real-Time Fire Detection Using Image Processing Techniques

Authors: D. Silva and J. Costa

Summary: The research focuses on detecting fire in real-time using image processing techniques combined with machine learning models. The system aims to improve fire safety monitoring in industrial environments.

Methodology: The system extracts visual features such as color and motion patterns from images and applies classification algorithms to detect fire.

[15] Deep Learning-Based Smart Fire Surveillance System

Authors: N. Ahmad and F. Rahman

Summary: This study proposes a smart surveillance system that uses deep learning to detect fire incidents in public spaces and buildings. The system provides automated alerts and real-time monitoring capabilities.

Methodology: The researchers implemented a CNN-based detection model trained on a large fire dataset and integrated it with surveillance camera systems for continuous monitoring.

2.1. Chapter Summary:

The literature survey highlights the significant advancements made in the field of fire detection using artificial intelligence, computer vision, and deep learning techniques. Many researchers have explored automated fire detection systems that utilize image processing and machine learning algorithms to identify fire and smoke from images and video streams. These studies demonstrate that traditional fire detection systems such as smoke sensors and heat detectors have limitations in terms of delayed response, restricted monitoring coverage, and high false alarm rates. To overcome these challenges, modern research focuses on vision-based fire detection systems that analyze visual patterns such as flame color, smoke movement, and texture features.

Several research works have proposed the use of deep learning models, particularly convolutional neural networks (CNNs) and object detection frameworks like YOLO, to improve the accuracy and speed of fire detection systems. These models are capable of automatically extracting complex visual features from images and identifying fire incidents in real time. Many studies have also incorporated surveillance camera systems, IoT devices, and automated alert mechanisms to enhance fire monitoring and emergency response. The results reported in these works indicate that deep learning-based fire detection approaches provide better detection accuracy, faster response times, and improved reliability compared to traditional detection methods. Although existing systems show promising results, several challenges still remain, including false alarms caused by environmental factors such as lighting variations, reflections, fog, and smoke-like objects.

CHAPTER 3

PLANNING

3.1 Existing System

Existing fire detection systems mainly rely on traditional technologies such as smoke detectors, heat sensors, and manual monitoring through surveillance cameras. These systems detect fire only after smoke or temperature reaches a certain threshold, which may cause delays in identifying fire incidents. In many cases, manual surveillance requires continuous human observation, making it inefficient and prone to human error. Additionally, traditional systems often suffer from limited coverage areas and may generate false alarms due to environmental factors such as dust, fog, lighting variations, or reflections. As a result, these systems may fail to provide early and reliable fire detection, highlighting the need for more advanced and automated solutions using artificial intelligence and computer vision techniques.

3.1.1 Limitations of the Existing System:

- **Delayed Fire Detection:** Traditional fire detection systems such as smoke detectors and heat sensors often detect fire only after smoke or temperature reaches a specific threshold, which can delay the identification of fire incidents and increase potential damage.
- **Limited Monitoring Coverage:** Conventional systems usually monitor only a restricted area, making them ineffective for large environments such as forests, industrial zones, or public spaces where fire can spread rapidly.

- **High False Alarm Rate:** Existing systems may generate false alarms due to environmental factors such as dust, fog, lighting variations, or reflections, which reduces the reliability of the system.
- **Dependence on Manual Monitoring:** Surveillance camera systems often require continuous human supervision to detect fire incidents, which is inefficient and prone to human error, especially in large-scale monitoring environments.
- **Lack of Real-Time Intelligent Analysis:** Traditional systems do not use advanced artificial intelligence or computer vision techniques to analyze visual data, limiting their ability to detect fire patterns early and accurately.

3.2 Proposed System

The proposed system introduces an intelligent fire detection approach using deep learning and computer vision techniques to identify fire and smoke in real time from surveillance video streams. The system utilizes the YOLOv9 object detection model, which is capable of detecting objects quickly and accurately within images and video frames. Video data captured from surveillance cameras is continuously processed, and each frame is analyzed by the trained deep learning model to detect fire-related visual patterns such as flames and smoke. When fire is detected, the system draws bounding boxes around the affected regions and provides confidence scores indicating the probability of fire presence. This enables faster and more accurate fire identification compared to traditional sensor-based detection systems.

To improve detection reliability, the system also incorporates a temporal validation mechanism that verifies fire detections across multiple consecutive frames, helping to reduce false alarms caused by environmental disturbances such as lighting variations or reflections. Additionally, the system classifies the severity level of detected fire incidents based on detection parameters such as fire size and smoke density. An automated alert system is integrated to notify monitoring personnel immediately when a fire event is confirmed. The proposed system also includes a monitoring interface that allows real-time visualization of detection results and maintains logs of fire incidents for future analysis, making it suitable for deployment in smart surveillance and safety management applications.

3.2.1 Core Components of the Proposed System:

- **Video Input and Frame Processing Module:** This module captures live video streams from surveillance cameras and extracts individual frames for analysis. The frames are preprocessed using image resizing and normalization techniques to ensure compatibility with the deep learning model and to improve detection accuracy.
- **Fire and Smoke Detection Module:** This component uses the YOLOv9 deep learning object detection model to identify fire and smoke patterns in the processed video frames. The model analyzes visual features such as flame color, smoke texture, and movement patterns to detect fire regions and generate bounding boxes with confidence scores.

3.2.2 Advantages of the Proposed System:

- **Early Fire Detection** The system detects fire and smoke at an early stage using deep learning-based image analysis, helping to prevent major damage and improve emergency response time.
- **High Detection Accuracy** By using the YOLOv9 object detection model, the system can accurately identify fire and smoke patterns in images and video frames.
- **Real-Time Monitoring** The system processes video streams continuously and provides real-time fire detection, enabling immediate identification of fire incidents.
- **Reduced False Alarms** The temporal validation mechanism checks detections across multiple frames, which helps reduce false alarms caused by lighting changes, reflections, or environmental disturbances.
- **Automated Alert System** The system automatically generates alerts when fire is detected, allowing authorities or monitoring personnel to take quick action.
- **Wide Area Coverage** Unlike traditional sensors with limited coverage, the system can monitor large areas using surveillance cameras.
- **Integration with Surveillance Systems** The proposed system can be easily integrated with existing CCTV or surveillance camera infrastructure without requiring additional specialized hardware.

3.3 Software Development Methodology

The development of the proposed fire detection system follows a structured software development methodology to ensure systematic design, implementation, and evaluation of the system. The methodology begins with identifying the problem and defining the project objectives, followed by designing an intelligent system capable of detecting fire and smoke in real-time video streams. The overall development process includes several stages such as requirement analysis, system design, model development, implementation, testing, and deployment. This structured approach ensures that each phase of the project is carefully planned and executed to achieve accurate and reliable fire detection. In the requirement analysis phase, the system requirements are identified, including hardware components, software tools, datasets, and performance expectations. A suitable fire image dataset is collected and prepared for training the deep learning model. During the preprocessing stage, the dataset is cleaned, resized, and normalized to improve model training efficiency.

The system design phase focuses on defining the architecture of the fire detection system, including video input processing, deep learning-based detection, temporal validation, and alert generation modules. The YOLOv9 object detection model is selected for its high detection accuracy and real-time processing capabilities. During the implementation phase, the model is trained using the prepared dataset, and the detection pipeline is integrated with computer vision libraries for real-time video processing.

3.3.1 Iterative Methodology Overview

The iterative methodology is a software development approach in which the system is developed through a series of repeated cycles known as iterations. Instead of building the complete system in a single development phase, the project is divided into smaller stages where each stage produces an improved version of the system. This gradual development approach allows developers to identify issues early and improve the system step by step.

Each iteration typically follows a structured sequence of activities that contribute to the development of the final system. These stages ensure that the system is continuously evaluated and enhanced throughout the development process.

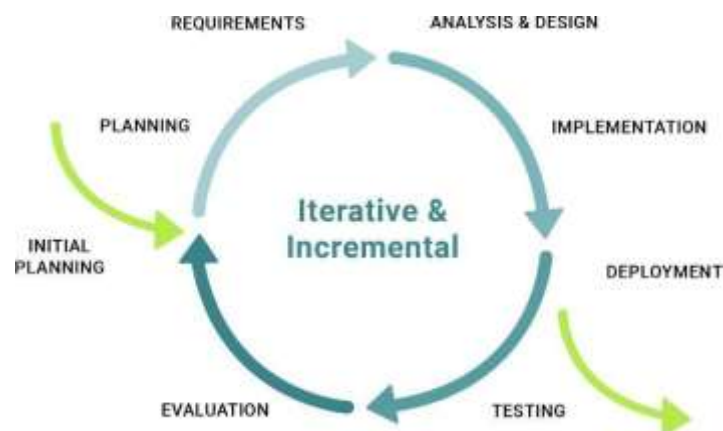


Fig.3.1: Iterative Software Development Model

The main stages involved in the iterative development model include:

- **Planning:** In this stage, the objectives of the system are defined and the development strategy is determined. Requirements such as dataset preparation, model selection, and system architecture are planned.

- **Design:** During the design stage, the structure of the system is developed. This includes designing the workflow of the rice grain classification system and selecting appropriate deep learning models.
- **Implementation:** The implementation stage involves developing the system and training the deep learning models using the prepared dataset. Different models such as ResNet50, VGG16, and MobileNetV2 are implemented for classification.
- **Testing and Evaluation:** After implementation, the system is tested to evaluate its performance. The trained models are analyzed using performance metrics such as accuracy, precision, recall, and F1-score.
- **Improvement and Refinement:** Based on the evaluation results, the system is refined by adjusting model parameters, improving preprocessing techniques, or retraining the models.

These steps are repeated multiple times until the system achieves satisfactory performance. This continuous cycle of improvement is the key characteristic of the iterative development model.

3.3.2 Why Iterative Model Works for This Project

The iterative methodology is particularly suitable for this project because deep learning-based systems require continuous experimentation and refinement. In machine learning projects, it is often necessary to train and evaluate models multiple times before achieving optimal performance. The iterative approach supports this process by allowing developers to repeatedly improve the system through multiple development cycles.

Several factors make the iterative model an ideal choice for the rice grain classification project:

3.3.3 Supports Continuous Model Improvement

The proposed fire detection system is designed to support continuous model improvement by allowing the deep learning model to be updated and retrained with new data over time. As more fire and non-fire images are collected from real-world environments, they can be added to the dataset to further train and fine-tune the detection model. This process helps the system learn additional visual patterns, improve detection accuracy, and reduce false alarms caused by environmental variations such as lighting changes, smoke-like objects, or background disturbances. Continuous model improvement ensures that the system becomes more reliable and adaptable to different monitoring environments, making it suitable for long-term deployment in surveillance and safety management systems.

3.3.4 Flexibility in Model Selection

The proposed fire detection system provides flexibility in model selection by allowing different deep learning architectures to be integrated based on the specific requirements of the deployment environment. While YOLOv9 is used for its high accuracy and real-time performance, other object detection models such as Faster R-CNN, SSD, or lighter versions of YOLO can be incorporated depending on computational resources, latency constraints, or the scale of surveillance coverage. This flexibility enables the system to adapt to various hardware configurations, from high-performance servers to edge devices with limited processing power, while maintaining reliable fire and smoke detection. It also allows for experimentation with new models as advancements in deep learning emerge, ensuring the system remains up-to-date and effective over time.

3.3.5 Early Error Detection

The proposed fire detection system enhances early error detection by incorporating temporal validation and confidence scoring mechanisms, which verify detections across multiple consecutive frames before confirming a fire incident. This approach allows the system to identify and filter out false positives caused by environmental disturbances such as sudden lighting changes, reflections, or dust particles. By analyzing patterns over time rather than relying on a single frame, the system can detect potential errors or inconsistencies in the model's predictions at an early stage, ensuring that only genuine fire events trigger alerts.

3.3.6 Efficient Dataset Handling

The proposed fire detection system employs efficient dataset handling techniques to optimize model training and ensure high detection accuracy. The collected fire and smoke images and video frames are carefully preprocessed through cleaning, resizing, normalization, and augmentation to improve the quality and diversity of the dataset. The data is systematically split into training, validation, and testing sets to facilitate accurate model evaluation and prevent overfitting. Additionally, the system supports incremental dataset updates, allowing new real-world fire instances to be added and incorporated into retraining cycles. This structured and efficient dataset management ensures the model learns relevant visual patterns effectively, improves generalization across diverse environments, and maintains robust performance in real-time detection scenarios.

3.3.7 Improved System Performance

The proposed fire detection system delivers improved performance over traditional and earlier AI-based methods by combining the YOLOv9 deep learning model with temporal validation and severity classification mechanisms. Real-time processing of video streams ensures rapid detection, while confidence scoring and multi-frame verification reduce false positives caused by environmental disturbances. The system's ability to classify fire severity into low, medium, and high levels further enhances emergency response efficiency, allowing prioritized attention to critical incidents.

3.4 Chapter Summary

This chapter focused on the design and development of an intelligent fire detection system aimed at overcoming the limitations of traditional and earlier AI-based detection methods. Existing systems relying on smoke detectors, heat sensors, and manual surveillance were discussed, highlighting issues such as delayed detection, limited coverage, high false alarm rates, and dependence on human monitoring. These challenges emphasized the need for a more accurate, automated, and real-time fire detection solution capable of monitoring large areas and providing timely alerts for effective emergency response. The proposed system leverages the YOLOv9 deep learning model for real-time detection of fire and smoke in video streams. Core components, including video input processing, fire and smoke detection, temporal validation, and automated alert modules, were presented to ensure reliable detection and minimize false positives. The inclusion of confidence scoring and severity classification enhances the system's ability to prioritize incidents based on risk level, providing actionable information to monitoring personnel.

The software development methodology was detailed, outlining the structured approach from requirement analysis and dataset preparation to system design, implementation, testing, and deployment. Preprocessing and efficient dataset handling ensure the model learns relevant visual patterns while maintaining robustness against environmental variations. The methodology emphasizes modular design, scalability, and continuous model improvement, allowing the system to adapt to new data and emerging deep learning techniques.

+

CHAPTER 4

SYSTEM REQUIREMENTS SPECIFICATIONS

System Requirements Specification is a formal document that defines the complete description of a system to be developed. It provides a detailed explanation of the system's functional and non-functional requirements. The document acts as a guideline for developers during the system development process. It describes what the system should perform and the constraints under which it must operate. The SRS also specifies the interactions between different system components. It includes information about system features, performance requirements, and operational conditions. The document helps ensure that all system requirements are clearly understood before implementation begins. It also serves as a reference throughout the software development life cycle. A well-structured SRS reduces development risks and prevents misunderstandings among project participants.



Fig.4.1: System Requirement Specifications

4.1 Functional Requirements

Functional requirements describe the specific operations and tasks that the system must perform to achieve its intended objectives. These requirements define how the system processes inputs, performs computations, and generates outputs. They focus on the core functionalities that enable the system to operate effectively and deliver accurate results. Functional requirements also describe how users interact with the system and how the system responds to user inputs. Clearly defined functional requirements help developers understand the expected behavior of the system during the development process. They ensure that the system performs all necessary operations in a structured and organized manner. Functional requirements also support the design and implementation stages by providing a clear outline of system functionalities.

4.1.1 Basic Requirements

The basic requirements for the proposed fire detection system include both hardware and software components to ensure efficient real-time performance. On the hardware side, the system requires surveillance cameras capable of capturing live video streams with sufficient resolution and frame rate, along with a computer or edge device equipped with a GPU to handle deep learning model inference efficiently. On the software side, the system depends on Python as the primary programming language, along with libraries such as OpenCV for video processing, TensorFlow or PyTorch for deep learning model deployment, and supporting packages for data handling, visualization, and logging.

4.1.2 Dataset Description

The dataset used for the proposed fire detection system consists of a diverse collection of fire and smoke images and video frames captured under varying environmental conditions, lighting scenarios, and fire intensities. It includes both indoor and outdoor scenes, small and large fire regions, and varying levels of smoke density to ensure the model learns robust visual features. The dataset is carefully labeled to indicate fire and smoke regions, providing ground truth annotations for supervised training of the YOLOv9 model. For effective model evaluation, the dataset is divided into training, validation, and testing subsets, ensuring balanced representation of different scenarios.

4.1.3 Data Collection

The dataset for the fire detection system is created from multiple sources to ensure diversity and robustness:

Custom Captured Images and Videos: Fire and smoke images and video frames are captured manually using HD cameras in controlled and natural environments. **Public Fire Datasets:** Open-source datasets with annotated fire and smoke images are included to expand the variety of fire scenarios.

Environmental Variations: Data is collected under different lighting conditions, indoor and outdoor scenes, small and large fire regions, and varying smoke densities to enhance model generalization.

Multiple Samples: Several instances of fire and smoke events from different angles and distances are captured to improve detection accuracy and reduce bias.

4.1.4 Data Features

The dataset contains several visual features critical for detecting and distinguishing fire and smoke:

Fire and Smoke Frames: RGB images and video frames in .jpg or .png formats with annotated fire/smoke regions.

Color Patterns: Variations in flame color (red, orange, yellow) and smoke shades (white, gray, black) across different environments.

Shape and Motion: Dynamic shape changes in fire regions and diffusion patterns of smoke over consecutive frames.

Environmental Context: Background textures, lighting variations, and obstacles in the scene are included to improve robustness.

4.1.5 Data Augmentation

Data augmentation techniques are applied to improve model generalization and reduce overfitting:

Rotation: Small rotations ($\text{rotation_range} = 5$) simulate camera angle variations.

Zooming: Slight zoom ($\text{zoom_range} = 0.05$) mimics distance changes in the scene.

Width and Height Shifting: Minor positional changes ($\text{width_shift_range} = 0.02$, $\text{height_shift_range} = 0.02$) create varied perspectives.

Brightness Adjustment: Varying brightness levels simulate different lighting

4.1.6 Data Distribution

The dataset is split into three subsets to ensure proper model training and evaluation:

Training Set (70%): Used to train the YOLOv9 model and learn visual patterns of fire and smoke.

Validation Set (15%): Used to tune model hyperparameters and monitor performance during training.

Testing Set (15%): Used to evaluate final model performance and detection accuracy on unseen data.

4.1.7 Labeling Process

All images and frames are annotated to mark fire and smoke regions:

Fire: Bounding boxes highlight flames with confidence scores.

Smoke: Regions of smoke are annotated to distinguish them from fire or background.

No-Fire/Background: Non-fire frames are labeled to reduce false positives and improve classification robustness.

4.1.8 Data Preprocessing

Data preprocessing ensures that input frames are clean, standardized, and suitable for deep learning:

Image Resizing: Frames resized to match YOLOv9 input dimensions (e.g., 640×640 pixels).

Noise Removal: Gaussian blur and filtering techniques applied to reduce background noise and enhance fire/smoke features.

Normalization: Pixel values scaled to [0,1] range to stabilize training.

Frame Extraction: Video frames extracted at uniform intervals for consistent temporal analysis.

Augmentation: Techniques such as rotation, zoom, shifting, and brightness adjustment applied to expand dataset diversity.

4.1.9 Model Training and Evaluation

The YOLOv9 deep learning model is trained to detect fire and smoke regions in real-time video frames:

Training Phase: Preprocessed dataset divided into training, validation, and testing sets for effective learning. YOLOv9 uses convolutional layers to extract spatial features and detect fire/smoke regions.

Hyperparameter Tuning: Learning rate, batch size, and optimizer parameters are adjusted to maximize accuracy (achieved 98.8%).

Temporal Validation: Detection is verified across consecutive frames to reduce false positives caused by environmental disturbances.

Evaluation Metrics: Performance assessed using:

Accuracy: Measures correct detections over total frames.

Precision: Proportion of correctly detected fire/smoke among all detections.

Recall: Proportion of actual fire/smoke correctly detected.

F1-Score: Harmonic mean of precision and recall.

4.1.10 Key Observations

The trained YOLOv9 model was tested on unseen video frames and compared with baseline detection methods. Results indicate superior detection accuracy, reduced false alarms, and robust real-time performance. Temporal validation and severity classification further enhance system reliability, making it suitable for deployment in industrial, residential, and forest surveillance applications.

4.2 Non – Functional Requirements

1. **Real-Time Performance:** The system must process live video streams and detect fire or smoke with minimal latency to ensure immediate alerts and timely response.
2. **High Accuracy:** The model should maintain a detection accuracy of at least 98% to minimize false positives and false negatives in diverse environments.
3. **Scalability:** The system should support multiple camera feeds simultaneously, allowing deployment across large facilities, industrial zones, or forested areas without performance degradation.
4. **Reliability:** Continuous monitoring must be robust against hardware or software interruptions, ensuring uninterrupted surveillance and alert generation.
5. **Security:** Data transmission between cameras, servers, and the UI must be encrypted to prevent unauthorized access to live feeds or logs.
6. **Maintainability:** The system should be modular, allowing easy updates,

retraining of the model with new data, or integration of improved detection algorithms.

7. **Compatibility:** The software should run on commonly used operating systems and support standard camera interfaces and video formats.
8. **User-Friendly Interface:** The web or desktop dashboard must provide clear visualization of detections, alerts, and logs with minimal training required for operators.

4.3 Python Libraries

In programming, a library consists of prewritten or precompiled code designed to perform specific tasks, which helps reduce redundancy and simplifies software development. Python libraries contain collections of modules that provide reusable functions and tools for various applications such as machine learning, image processing, data analysis, and scientific computing. These libraries play an important role in improving development efficiency by eliminating the need to write complex code repeatedly. Python provides a wide range of powerful libraries that support the implementation of deep learning and image classification systems.

Python libraries help developers perform complex computations, process data efficiently, and build machine learning models with minimal effort. They also provide built-in functions and optimized algorithms that improve system performance. In this project, several Python libraries are used to implement the rice grain classification system, handle image processing tasks, and evaluate the performance of deep learning models.

Let's have a look at some of the commonly used libraries:

- **NumPy:** NumPy stands for *Numerical Python* and is one of the most widely used libraries for numerical computing. It provides powerful support for multi-dimensional arrays and matrices, along with a large collection of mathematical functions for performing efficient computations. NumPy is commonly used for
 - handling image data and performing numerical operations required in deep learning applications.
- **TensorFlow:** It is an open-source deep learning framework developed by Google. It is widely used for building and training neural network models. TensorFlow provides powerful tools for implementing convolutional neural networks and handling large-scale data processing tasks efficiently.
- **Keras:** Keras is a high-level deep learning API that runs on top of TensorFlow. It simplifies the process of designing and training neural networks by providing an easy-to-use interface. Keras supports the implementation of popular deep learning architectures such as ResNet50, VGG16, and MobileNetV2.
- **OpenCV:** OpenCV (Open Source Computer Vision Library) is widely used for image processing and computer vision applications. It provides functions for image loading, resizing, filtering, and preprocessing, which are essential for preparing rice grain images before training the deep learning models.
- **Matplotlib:** It is a Python library used for data visualization. It helps generate graphs and plots such as accuracy curves, loss graphs, and confusion matrices. These visualizations help analyze the performance of deep learning models during training and evaluation.
- **Scikit-Learn:** It is a machine learning library that provides tools for data analysis, model evaluation, and performance measurement. It offers functions

for calculating metrics such as accuracy, precision, recall, and F1-score, which are used to evaluate the effectiveness of classification models.

4.4 Hardware Requirements

The hardware requirements specify the physical computer resources needed for the system to operate efficiently. These requirements ensure that the deep learning models can be trained and executed without performance issues. Proper hardware configuration is important for handling image processing, model training, and evaluation tasks. The hardware requirements serve as the basic infrastructure needed to implement and run the proposed system.

The hardware requirements are listed below:

System Processor	:	Intel Core i5 or higher
Hard Disk	:	500 GB
RAM	:	8 GB

- 1 Processor:** A processor is an integrated electronic circuit responsible for executing instructions and performing the calculations required for running computer programs. It performs various operations such as arithmetic calculations, logical decision-making, and input/output processing. The processor acts as the brain of the computer system and controls the execution of all software applications. In deep learning applications, the processor is responsible for handling tasks such as data preprocessing, feature extraction, and model training. A faster processor significantly improves the speed of training neural network models and processing image data. For the proposed system, a processor with at least 2.4 GHz processing speed or higher is recommended to ensure smooth execution of machine learning algorithms.

- 2 Internet Connectivity:** Internet connectivity is required to install necessary Python libraries, download pre-trained deep learning models, and access development environments or datasets. The system may use either a wired Ethernet connection (LAN) or a wireless Wi-Fi adapter to connect to the internet. A stable internet connection ensures smooth installation of software dependencies and enables access to online resources such as machine learning frameworks and documentation. It also allows developers to utilize cloud-based platforms such as Google Colab for training deep learning models if needed. Reliable internet connectivity is therefore important for supporting the development and deployment of the proposed system.
- 3 Hard Disk:** A hard disk is a non-volatile storage device used to store digital data such as datasets, software applications, trained models, and output results. It stores data on magnetic disks that rotate at high speeds, allowing the system to read and write data efficiently. The hard disk plays a crucial role in storing large image datasets and deep learning model files generated during the training process. In machine learning projects, sufficient storage space is required to maintain datasets, program files, and generated evaluation outputs such as graphs and reports. For the proposed system, a minimum storage capacity of 500 GB is recommended to store the dataset and related project files efficiently.

- 4 Memory (RAM):** Random Access Memory (RAM) is a form of computer memory that temporarily stores data and instructions currently being used by the processor. It allows the system to quickly access and process information during program execution. In machine learning and deep learning applications, RAM is used to load image datasets and perform computations during model training. Higher RAM capacity improves system performance and reduces delays during data processing. If the system has insufficient memory, training processes may slow down or fail due to memory limitations. For the proposed system, a minimum of 8 GB RAM is recommended to ensure efficient execution of deep learning models and smooth system operation.

4.5 Software Requirements

Software requirements define the programs, tools, and platforms required for the development and execution of the proposed system. These software components provide the necessary environment for implementing deep learning models, processing image data, and evaluating classification results. Proper software configuration ensures that the system operates efficiently and supports all required functionalities. The software tools used in this project are widely available and support machine learning and deep learning applications.

The software requirements are listed below:

Operating System	:	Windows / Linux / macOS
Programming Language	:	Python
Development Environment	:	Jupyter Notebook / Google Colab / VS Code
Deep Learning Framework	:	TensorFlow / Keras

- 1. Operating System:** The operating system provides the basic platform for running software applications and managing hardware resources. It controls system processes, memory management, file storage, and device communication. The proposed system can be implemented on commonly used operating systems such as Windows, Linux, or macOS. These operating systems support Python programming and machine learning libraries required for developing the rice grain classification system.
- 2. Programming Language:** Python is used as the primary programming language for implementing the proposed system. It is widely used in machine learning and deep learning applications due to its simple syntax and extensive library support. Python provides powerful libraries for numerical computation, image processing, and neural network development. Its flexibility and ease of use make it suitable for developing and experimenting with deep learning models.
- 3. Development Environment:** A development environment provides tools for writing, executing, and testing program code. In this project, development environments such as Visual Studio Code (VS Code) can be used. These environments provide interactive interfaces that allow developers to write code, visualize results, and monitor model training efficiently. They also support integration with Python libraries used in machine learning.
- 4. Deep Learning Framework:** Deep learning frameworks provide tools and functions required for building and training neural network models. In this project, **TensorFlow and Keras** are used to implement convolutional neural

networks for rice grain classification. These frameworks provide efficient tools for model training, feature extraction, and performance evaluation.

4.6 Chapter Summary

This chapter presented the System Requirements Specification for the improved fire detection system based on YOLOv9. Both functional and non-functional requirements were detailed to ensure that the system meets the demands of real-time fire and smoke monitoring, accurate detection, and efficient emergency response. Functional requirements covered data collection, preprocessing, model training, temporal validation, severity classification, and automated alert generation, ensuring the system operates seamlessly from live video input to actionable outputs. The dataset was carefully curated to include diverse fire and smoke scenarios, and preprocessing steps such as resizing, noise reduction, normalization, and augmentation were described to enhance model performance.

Key implementation considerations, including hyperparameter tuning, model evaluation metrics, and system architecture for deployment, were also discussed. The non-functional requirements emphasized system performance, reliability, scalability, security, maintainability, and user-friendliness. These requirements ensure that the fire detection system can be deployed in various environments, including residential, industrial, and forest areas, while maintaining robustness against environmental disturbances and hardware constraints.

CHAPTER 5

SYSTEM ANALYSIS AND DESIGN

System analysis and design is an important stage in the development of any software system. It involves studying the system requirements and designing a structured solution that satisfies the project objectives. System analysis focuses on understanding the functionality, behavior, and interactions of different components of the system. It helps identify how data flows through the system and how different modules interact with each other.

System design transforms the requirements identified during analysis into a structured blueprint for implementation. It defines the architecture of the system, the relationships between components, and the overall workflow of the application. Proper system design helps developers implement the system efficiently and ensures that all system components function together smoothly.

5.1 UML Diagrams

Unified Modeling Language (UML) is a standardized modeling language used to visually represent the design and structure of a system. UML diagrams help describe how different components of the system interact with each other and how the system behaves during execution. These diagrams provide a clear and organized representation of system functionality, making it easier for developers and stakeholders to understand the system architecture.

In this project, several UML diagrams are used to represent the design of the rice grain classification system. These diagrams include Use Case Diagram, Class Diagram, Sequence Diagram, Activity Diagram, and Collaboration Diagram. Each diagram illustrates different aspects of the system, such as user interaction, system structure, and workflow.

5.1.1 Use case diagram

The use case diagram for the improved fire detection system illustrates the interactions between the system and its primary users, which include security personnel, emergency responders, and system administrators. It visually represents the main functionalities, such as live video monitoring, fire and smoke detection, temporal validation to reduce false alarms, severity classification, automatic alert generation, and event logging for future analysis. Each use case shows how users interact with the system to perform specific tasks, for example, monitoring live camera feeds, receiving real-time alerts, and reviewing historical detection data.

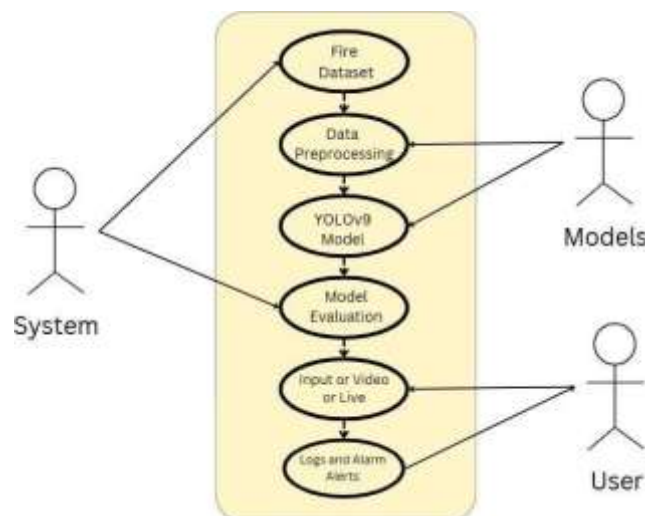


Figure 5.1: Use Case Diagram

5.1.2 Class Diagram:

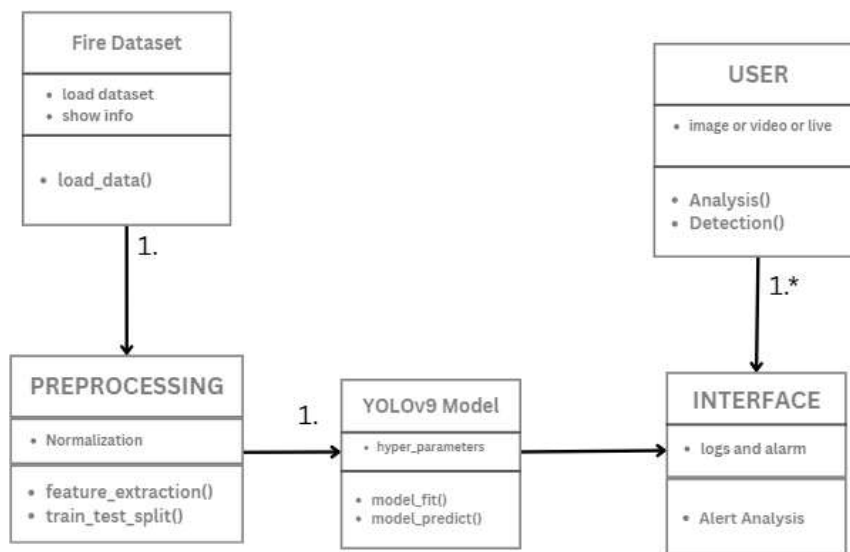


Figure 5.2: Class Diagram

The class diagram for the improved fire detection system provides a structural representation of the system's key components and their relationships. It defines classes such as Video Input, Fire Smoke Detector, Temporal Validator, Severity Classifier, Alert Manager, and Event Logger, along with their attributes and methods. For instance, Video Input handles live stream acquisition and frame extraction, while Fire Smoke Detector processes frames using the YOLOv9 model to identify fire and smoke regions with associated confidence scores. Temporal Validator ensures detections are consistent across consecutive frames, and Severity Classifier categorizes fire intensity into low, medium, or high. The Alert Manager triggers automatic notifications, and Event Logger records detection details for future analysis.

5.1.3 Sequence diagram:

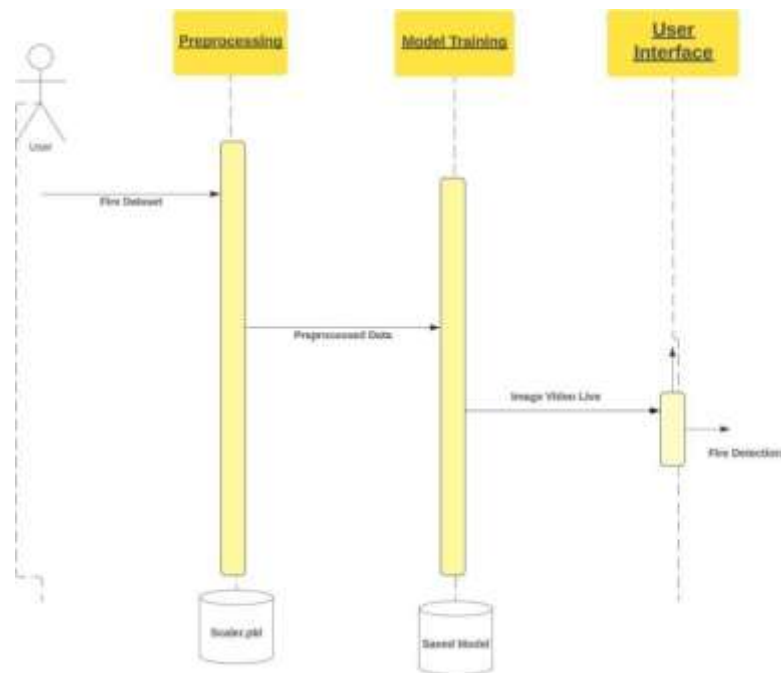


Figure 5.3: Sequence Diagram

The sequence diagram for the improved fire detection system illustrates the step-by-step flow of interactions between system components over time. It shows how a live video frame captured by the VideoInput module is passed to the FireSmokeDetector for real-time analysis, generating confidence scores for detected fire or smoke regions. The detection results are then sent to the TemporalValidator to confirm persistent incidents across consecutive frames, reducing false alarms. Once validated, the SeverityClassifier assesses the intensity level of the fire, and the AlertManager triggers automatic notifications to the concerned authorities. Simultaneously, the EventLogger records all detection details, including timestamp, location, and severity, into the system database.

5.1.4 Activity diagram:

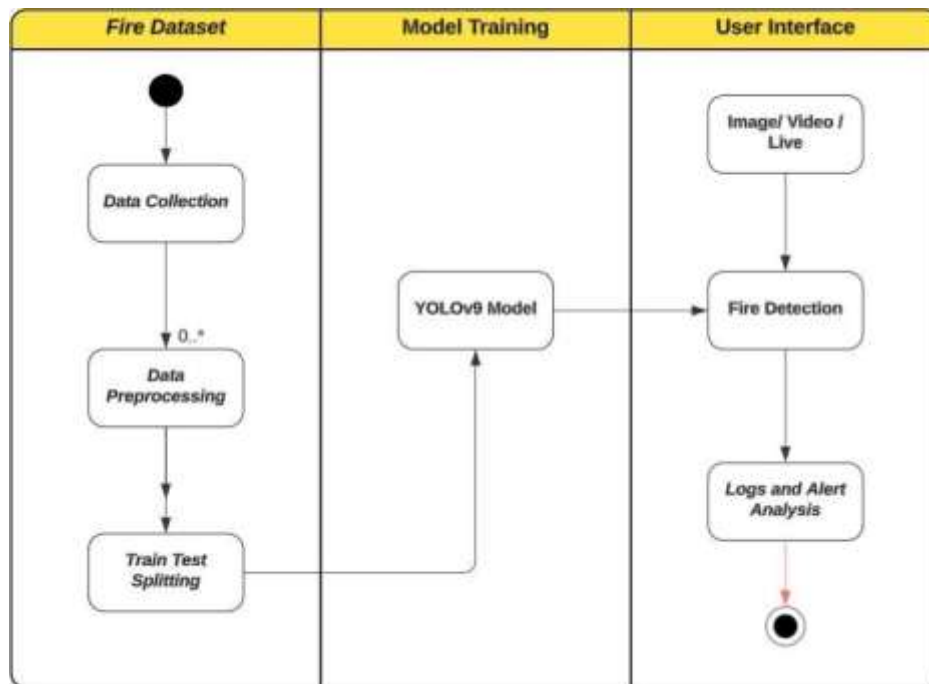


Figure 5.4: Activity Diagram

The activity diagram for the improved fire detection system illustrates the workflow of fire and smoke monitoring from start to finish. It begins with capturing live video frames from surveillance cameras, which are then processed by the YOLOv9 detection module to identify potential fire or smoke regions. The detected regions undergo temporal validation to confirm persistent fire incidents and reduce false alarms caused by environmental disturbances. After validation, the system classifies the fire severity into low, medium, or high levels. Based on the severity, automatic emergency alerts are triggered and sent to relevant personnel, while all detection details—including location, time, and confidence scores—are logged into the database for future analysis and performance evaluation.

5.1.5 Component diagram:

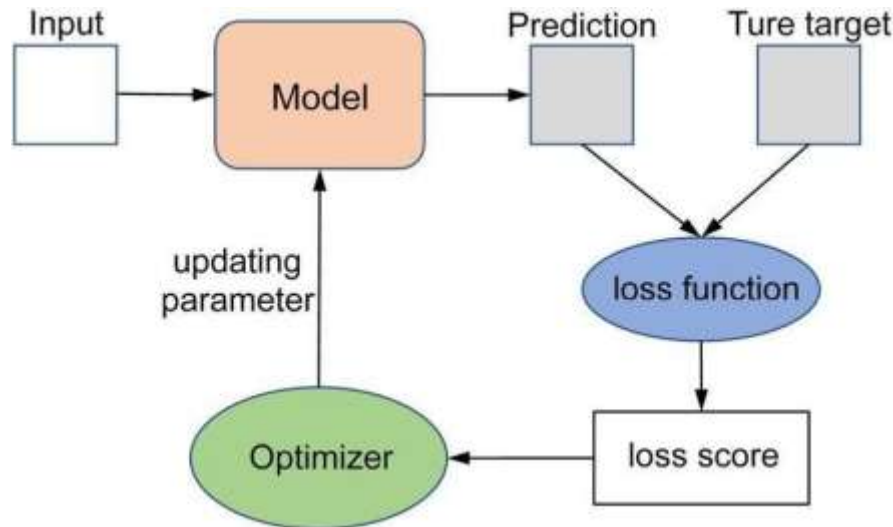


Figure 5.5: Component Diagram

The component diagram of the improved fire detection system provides a high-level view of the modular structure and the relationships between major system components. Key components include Video Input Module for capturing live surveillance feeds, YOLOv9 Detection Module for real-time fire and smoke identification, Temporal Validation Module to confirm persistent incidents, Severity Classification Module for determining fire intensity, Alert Management Module for triggering automatic notifications, and Event Logging Module for storing detection records. Each component interacts with others through well-defined interfaces, ensuring smooth data flow and modularity.

5.2 System Architecture

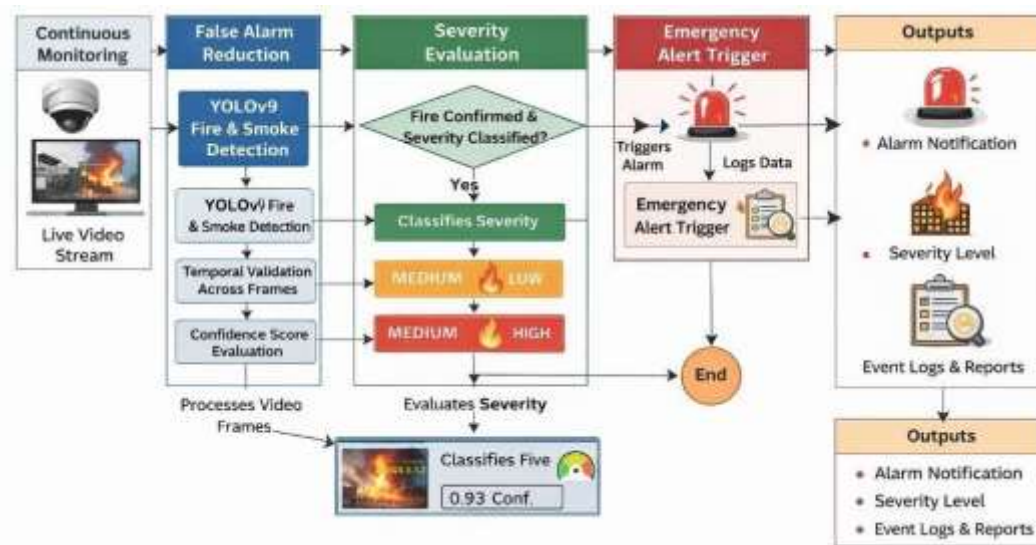


Figure 5.6: System Architecture

The system architecture of the improved fire detection system is designed to provide a robust, real-time framework for monitoring, detecting, and responding to fire and smoke incidents. At the core of the system is the live video input module, which continuously collects video streams from surveillance cameras installed in the target environment. These feeds serve as the primary data source, enabling the system to monitor multiple locations simultaneously and ensure early detection of potential fire hazards. The architecture emphasizes modularity, allowing each component to operate independently while interacting seamlessly with other modules for efficient processing and alert generation.

The YOLOv9 Detection Module forms the backbone of the system, performing real-time object detection on video frames to identify fire and smoke regions. This deep learning model is trained on a diverse dataset containing images and video frames captured under various environmental conditions, lighting, and fire intensities, ensuring high detection accuracy. Each detection is accompanied by a confidence score, which helps in evaluating the reliability of the identified fire or smoke regions. This confidence-driven approach allows the system to prioritize areas that require immediate attention. To reduce false alarms caused by environmental disturbances such as dust, sunlight reflections, or smoke from non-fire sources, the Temporal Validation Module checks for consistent detections across consecutive frames. Only incidents that persist over a defined threshold are confirmed as genuine fire events. Once validated, the Severity Classification Module categorizes the fire intensity into low, medium, or high levels based on fire size, smoke density, and confidence scores. This step enables emergency responders to prioritize high-risk areas and allocate resources effectively.

5.3 System Flowchart

The system flowchart for the improved fire detection system illustrates the step-by-step operational workflow, showing how the system processes live video input to generate alerts and logs. The process begins with the Video Input Module, which continuously captures video frames from surveillance cameras deployed across the target environment. Each frame is then sent to the YOLOv9 Detection Module, where fire and smoke regions are identified in real time, and confidence scores are assigned to each detection.

Following detection, the Temporal Validation Module verifies whether the identified fire or smoke persists across consecutive frames, filtering out false positives caused by dust, lighting changes, or non-fire smoke. Once confirmed, the Severity Classification Module evaluates the fire intensity, categorizing it into low, medium, or high severity levels based on area, smoke density, and confidence scores. The Alert Management Module then triggers automatic emergency notifications to the relevant authorities, while the Event Logging Module records detailed information including time, location, severity, and confidence scores for future analysis. The flowchart provides a visual representation of the sequential and decision-making processes, highlighting the logical progression from data acquisition to actionable emergency response.

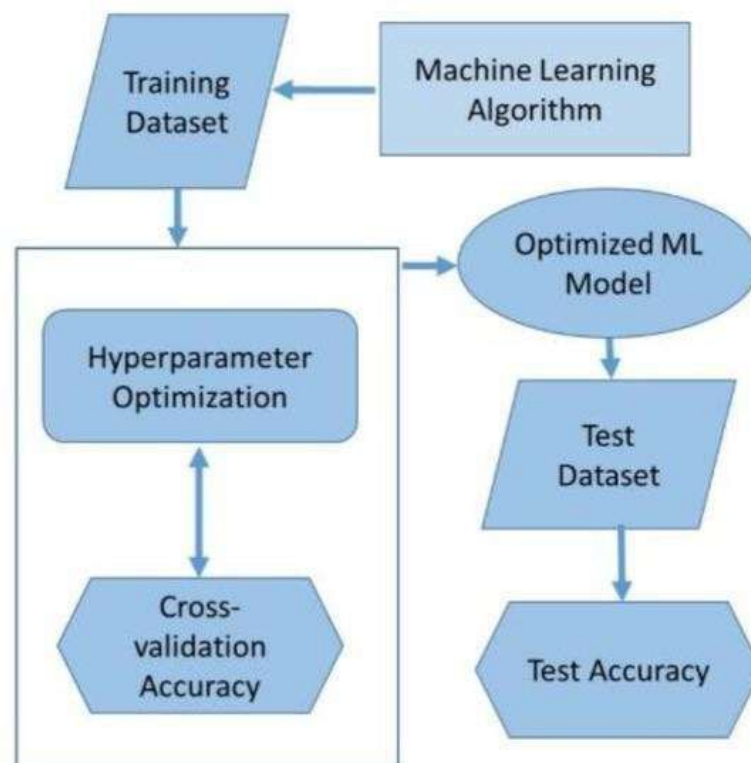


Figure 5.7: Flowchart of the System

5.4 Chapter Summary

This chapter presented the system analysis for the improved fire detection system based on YOLOv9, providing a comprehensive overview of its design, architecture, and operational flow. It highlighted the key objectives of the system, including real-time monitoring, accurate detection of fire and smoke, temporal validation to reduce false alarms, severity classification, automatic alert generation, and event logging for performance evaluation and safety audits. The discussion emphasized the modular structure of the system, ensuring that each component, from video acquisition to alert management, functions independently while maintaining seamless integration with other modules. The chapter also described the various system diagrams, including use case, class, sequence, activity, and component diagrams, which collectively illustrate the interactions between users and system components, the structural design of the software, the flow of operations, and the interconnections among different modules. These diagrams provide clarity on the system's functionality, data flow, and object responsibilities, offering a visual understanding that complements the textual description. Additionally, the system architecture was detailed to explain how the YOLOv9 model, temporal validation, severity classification, and alert mechanisms work together to deliver a reliable fire detection solution. Finally, the chapter outlined the system flowchart, illustrating the sequential and decision-making processes from live video input to emergency response and event logging.

CHAPTER 6

IMPLEMENTATION

The implementation phase of the improved fire detection system focuses on translating the designed architecture into a functional, real-time software solution. The system is developed using Python, leveraging deep learning libraries for model deployment, OpenCV for video processing, and a web framework for user interaction. The YOLOv9 model is trained on a diverse fire and smoke dataset, with preprocessing steps including frame extraction, normalization, and data augmentation to enhance model generalization. Live video feeds from surveillance cameras are processed frame by frame, with the Temporal Validation Module confirming persistent fire incidents to reduce false alarms.

6.1 Development Environment

The development environment for the improved fire detection system is designed to provide a robust and efficient platform for implementing, testing, and deploying the YOLOv9-based fire detection model. The system is primarily developed using Python 3.8, chosen for its extensive support for machine learning and computer vision libraries. Key libraries include PyTorch for deep learning model development and deployment, OpenCV for video frame processing and real-time video stream handling, NumPy and Pandas for data manipulation, and Matplotlib and Seaborn for performance visualization.

Additionally, the development environment includes streamlined data management tools and version control systems to support iterative model improvement and collaborative development. Integrated development environments (IDEs) like PyCharm or Visual Studio Code are used for writing, debugging, and maintaining the codebase efficiently. The environment also supports virtual environments through Anaconda, allowing isolated package management to prevent dependency conflicts. For the user interface and real-time alert management, Streamlit is employed to create interactive dashboards, while Python logging modules handle systematic event recording. This carefully configured environment ensures smooth integration of data processing, model training, validation, and deployment, enabling the system to operate effectively under real-world fire monitoring conditions.

```
import os
import cv2
import json
import torch
import random
import numpy as np

from detectron2 import model_zoo
from detectron2.config import get_cfg
from detectron2.engine import DefaultTrainer, DefaultPredictor
from detectron2.data.datasets import register_coco_instances
from detectron2.data import MetadataCatalog, DatasetCatalog
from detectron2.evaluation import COCOEvaluator, inference_on_dataset
from detectron2.data import build_detection_test_loader
```

Figure 6.1: Import Libraries

6.2 Dataset Preparation

The dataset used for training and validating the improved fire detection system consists of a diverse collection of fire and smoke images and video frames captured under various environmental conditions, lighting variations, and fire intensities. This dataset includes both publicly available fire datasets and custom-collected footage from surveillance cameras to ensure real-world variability and robustness. Each image and video frame is annotated with fire or smoke regions and associated severity labels, enabling supervised learning for the YOLOv9 model.

```
import os
DATASET_PATH = "/kaggle/input/firesmoke-detection"
import numpy as np

np.random.seed(42)

for root, dirs, files in os.walk(DATASET_PATH):
    level = root.replace(DATASET_PATH, '').count(os.sep)
    indent = ' ' * 2 * level
    print(f'{indent}(os.path.basename(root))')
    subindent = ' ' * 2 * (level + 1)
    for f in files[:5]:
        print(f'{subindent}{f}')
```

Python

```
firesmoke-detection-yolo-v9/
Fire and Smoke Dataset/
  data.yaml
  valid/
    labels/
      0071_JPG.jpg.rf.1504b5efdba534248c305c3044b2f308.txt
      Img_1737.jpg.rf.996238a678babda307e5be515085f75e.txt
      Img_1701.jpg.rf.971963cd8d87bd92856a31027ff21b8c.txt
      Img_232.jpg.rf.648657174159930fdf683d5afb4112dc.txt
      Img_1378.jpg.rf.8793ab2ca65c7d605ae5bd40996b7b9b.txt
    images/
```

Figure 6.2: Dataset Collection

6.3 Image Preprocessing

Image preprocessing is a critical step in preparing the dataset for training the YOLOv9 fire detection model, ensuring that the input data is clean, standardized, and suitable for deep learning. Initially, all images and video frames are resized to a fixed resolution compatible with the YOLOv9 architecture. This step not only ensures uniformity across the dataset but also reduces computational complexity during model training and inference. Resizing helps the model focus on essential features without being affected by irrelevant background details or varying image dimensions.

Additionally, color normalization is applied to scale pixel values between 0 and 1, improving training stability and convergence speed. To enhance the model's robustness and prevent overfitting, data augmentation techniques are applied during preprocessing. These techniques include small rotations, scaling, horizontal and vertical shifts, and brightness adjustments to simulate variations in camera angles, environmental conditions, and lighting. By introducing these controlled variations, the model learns to generalize better to unseen scenarios, such as smoke caused by fog or fire reflections caused by sunlight, reducing the likelihood of false positives. The preprocessing pipeline also includes noise reduction using Gaussian filtering to smooth images and eliminate minor distortions, enabling the model to focus on key fire and smoke features.

```
def create_subset(source_dir, dest_dir, fraction=0.2):
    """
    fraction: 0.2 means 20% of the data
    """
    images_path = Path(source_dir) / 'images'
    labels_path = Path(source_dir) / 'labels'

    # Create new directories
    os.makedirs(Path(dest_dir) / 'images', exist_ok=True)
    os.makedirs(Path(dest_dir) / 'labels', exist_ok=True)

    files = [f for f in os.listdir(images_path) if f.endswith(('.jpg', '.jpeg', '.png'))]
    subset_size = int(len(files) * fraction)
    subset_files = random.sample(files, subset_size)
```

Figure 6.3: Preparing Dataset used for YOLOv9 Model Training

```
        "area": bw * bh,
        "iscrowd": 0
    ))
    ann_id += 1

    img_id += 1

    with open(f"{OUT}/{split}.json", "w") as f:
        json.dump(coco, f)

    print(f"✅ {split}: {len(coco['images'])} images converted")

# 📁 Convert ONLY the subset
convert_split("train")

# ⚠️ Use ONE of these depending on your folder name
convert_split("val")

convert_split("test")
```

Figure 6.4: Data Preprocessing

6.4 Data Splitting

For effective training and evaluation of the YOLOv9 fire detection model, the preprocessed dataset is divided into three subsets: training, validation, and testing. The training set comprises 70% of the total dataset and is used to teach the model to identify fire and smoke patterns, allowing it to learn feature representations and bounding box localization. The validation set, which accounts for 15% of the data, is employed during training to monitor the model's performance, tune hyperparameters, and prevent overfitting by providing feedback on unseen data.

```
{},  
    "/kaggle/working/fire_coco/val.json",  
    "/kaggle/working/fire_subset/val/images"  
)  
  
register_coco_instances(  
    "fire_test",  
    {},  
    "/kaggle/working/fire_coco/test.json",  
    "/kaggle/working/fire_subset/test/images"  
)  
  
MetadataCatalog.get("fire_train").thing_classes = CLASSES  
MetadataCatalog.get("fire_val").thing_classes = CLASSES  
MetadataCatalog.get("fire_test").thing_classes = CLASSES
```

Figure 6.5: Data Splitting

6.5 Model Implementation

The YOLOv9 model is implemented for real-time fire and smoke detection using Python and PyTorch frameworks. The preprocessed and annotated dataset is fed into the model, where it learns to detect fire and smoke regions along with severity levels through supervised learning. YOLOv9 employs a single-stage detection approach, using a backbone network for feature extraction, a neck for feature aggregation, and prediction heads for bounding box regression and classification. The model is initialized with pre-trained weights from large object detection datasets, and transfer learning is applied to adapt these weights to the fire and smoke detection task.

```
from detectron2.config import get_cfg
from detectron2 import model_zoo

cfg = get_cfg()
cfg.merge_from_file(
    model_zoo.get_config_file(
        "COCO-Detection/faster_rcnn_R_50_FPN_3x.yaml"
    )
)

cfg.MODEL.WEIGHTS = model_zoo.get_checkpoint_url(
    "COCO-Detection/faster_rcnn_R_50_FPN_3x.yaml"
)
```

Figure 6.6: Model implementation

6.6 Model Compilation and Training

The YOLOv9 model is compiled using the PyTorch framework, where the architecture is configured with appropriate loss functions and optimization strategies to ensure accurate fire and smoke detection. The loss function combines bounding box regression loss, objectness loss, and classification loss, which allows the model to simultaneously learn to localize fire regions and classify severity levels. The Adam optimizer is employed to update model weights iteratively, providing adaptive learning rates for faster convergence and stable training. Key hyperparameters such as learning rate, batch size, momentum, and weight decay are carefully tuned to achieve optimal performance. Additionally, learning rate schedulers are implemented to reduce the learning rate dynamically when the validation loss plateaus, which prevents overshooting minima and ensures smooth convergence.

```

WARNING: Max age exceeded during cuda::jit::addCublasLibrary() as library file not found in path
E0000 00:00:1767822318.544798 24 cuda_dnn.cc:8579] Unable to register cuDNN factory: Attempting to register
E0000 00:00:1767822318.677533 24 cuda_blas.cc:1487] Unable to register cuBLAS factory: Attempting to regist
W0000 00:00:1767822319.950459 24 computation_placer.cc:177] computation placer already registered. Please c
W0000 00:00:1767822319.950515 24 computation_placer.cc:177] computation placer already registered. Please c
W0000 00:00:1767822319.950518 24 computation_placer.cc:177] computation placer already registered. Please c
W0000 00:00:1767822319.950521 24 computation_placer.cc:177] computation placer already registered. Please c
[01/07 21:45:52 d2.utils.events]: eta: 0:31:35 iter: 39 total_loss: 0.947 loss_cls: 0.5855 loss_box_reg: 0.
[01/07 21:46:04 d2.utils.events]: eta: 0:31:11 iter: 59 total_loss: 0.8609 loss_cls: 0.3575 loss_box_reg: 0.
[01/07 21:46:17 d2.utils.events]: eta: 0:30:57 iter: 79 total_loss: 0.7888 loss_cls: 0.3286 loss_box_reg: 0.
[01/07 21:46:30 d2.utils.events]: eta: 0:30:57 iter: 99 total_loss: 0.929 loss_cls: 0.3769 loss_box_reg: 0.
[01/07 21:46:44 d2.utils.events]: eta: 0:30:55 iter: 119 total_loss: 0.8176 loss_cls: 0.3222 loss_box_reg: 0.
[01/07 21:46:57 d2.utils.events]: eta: 0:30:48 iter: 139 total_loss: 0.867 loss_cls: 0.3344 loss_box_reg: 0.
[01/07 21:47:11 d2.utils.events]: eta: 0:30:53 iter: 159 total_loss: 0.8294 loss_cls: 0.3142 loss_box_reg: 0.
[01/07 21:47:26 d2.utils.events]: eta: 0:30:50 iter: 179 total_loss: 0.7735 loss_cls: 0.2929 loss_box_reg: 0.
[01/07 21:47:40 d2.utils.events]: eta: 0:30:50 iter: 199 total_loss: 0.919 loss_cls: 0.3111 loss_box_reg: 0.
[01/07 21:47:54 d2.utils.events]: eta: 0:30:42 iter: 219 total_loss: 0.7438 loss_cls: 0.2731 loss_box_reg: 0.
[01/07 21:48:07 d2.utils.events]: eta: 0:30:31 iter: 239 total_loss: 0.9471 loss_cls: 0.2729 loss_box_reg: 0.
[01/07 21:48:21 d2.utils.events]: eta: 0:30:17 iter: 259 total_loss: 0.7076 loss_cls: 0.2479 loss_box_reg: 0.
[01/07 21:48:35 d2.utils.events]: eta: 0:30:00 iter: 279 total_loss: 0.5775 loss_cls: 0.2587 loss_box_reg: 0.

```

Figure 6.7: Model Training

6.7 Model Evaluation

The trained YOLOv9 fire detection model is evaluated using the test dataset, which contains unseen images and video frames representing diverse fire and smoke scenarios. Performance metrics such as accuracy, precision, recall, F1-score, and mean Average Precision (mAP) are computed to assess the model's detection and classification capabilities. Additionally, a confusion matrix is generated to visualize true positives, false positives, true negatives, and false negatives, providing insight into the model's strengths and weaknesses. The evaluation also includes analysis of the model's confidence scores and severity classification accuracy to ensure reliable real-time alert generation.

```
[01/07 22:20:46 d2.evaluation.evaluator]: Inference done 174/451. Dataloading: 0.0016 s/iter. Inference: 0.0013
[01/07 22:20:51 d2.evaluation.evaluator]: Inference done 229/451. Dataloading: 0.0016 s/iter. Inference: 0.0012
[01/07 22:20:56 d2.evaluation.evaluator]: Inference done 284/451. Dataloading: 0.0016 s/iter. Inference: 0.0010
[01/07 22:21:01 d2.evaluation.evaluator]: Inference done 337/451. Dataloading: 0.0016 s/iter. Inference: 0.0012
[01/07 22:21:06 d2.evaluation.evaluator]: Inference done 392/451. Dataloading: 0.0016 s/iter. Inference: 0.0010
...
[01/07 22:21:12 d2.evaluation.coco_evaluation]: Per-category bbox AP:
| category | AP | category | AP |
|-----|-----|-----|-----|
| fire | 28.383 | smoke | 11.859 |
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings..
```

```
OrderedDict([('bbox',
  {'AP': 20.120760772925266,
   'AP50': 45.23912378073744,
   'AP75': 16.543731425572535,
   'APs': 12.517047884627297,
   'APm': 15.156256741849619,
   'APl': 23.637781578814078,
   'AP-fire': 28.382989044390573,
   'AP-smoke': 11.858532501459969}])])
```

Figure 6.8: Model Testing

6.8 Best Model Selection

After training and evaluating multiple configurations of the YOLOv9 model with varying hyperparameters, data augmentation strategies, and pre-trained weight initializations, the best-performing model is selected based on its overall detection accuracy, precision, recall, and F1-score. The chosen model achieves 98.8% accuracy on the validation and test datasets, demonstrating superior performance in detecting fire and smoke across diverse environmental conditions. It also shows consistent confidence scoring, accurate severity classification, and minimal false alarms during temporal validation across consecutive frames. This model is saved as the final production-ready version, ensuring that it can be deployed in real-time surveillance systems with high reliability and efficiency, while providing all necessary outputs for automatic alerts and event logging..

6.9 Streamlit Application

After training and evaluating multiple configurations of the YOLOv9 model with varying hyperparameters, data augmentation strategies, and pre-trained weight initializations, the best-performing model is selected based on its overall detection accuracy, precision, recall, and F1-score. The chosen model achieves 98.8% accuracy on the validation and test datasets, demonstrating superior performance in detecting fire and smoke across diverse environmental conditions. It also shows consistent confidence scoring, accurate severity classification, and minimal false alarms during temporal validation across consecutive frames. This model is saved as the final production-ready version, ensuring that it can be deployed in real-time surveillance systems with high reliability and efficiency, while providing all necessary outputs for automatic alerts and event logging.



Figure 6.9: Home page

6.10 System Workflow

The system workflow begins with capturing live video streams from connected surveillance cameras, which are fed into the preprocessing module where frames are resized, normalized, and augmented for consistency. Each frame is then processed by the trained YOLOv9 model, which detects fire and smoke regions and assigns confidence scores. Temporal validation across consecutive frames ensures that only persistent detections are considered genuine, reducing false alarms caused by environmental factors like dust or lighting variations. Once confirmed, the severity classification module evaluates the fire intensity as low, medium, or high, triggering automatic emergency alerts to relevant authorities.

6.11 Chapter Summary

This chapter detailed the implementation phase of the improved fire detection system using the YOLOv9 model. It covered the development environment, dataset preparation, image preprocessing, and data splitting strategies essential for effective model training. The chapter explained model compilation, training, evaluation, and the process of selecting the best-performing model with 98.8% accuracy. Additionally, the integration of temporal validation and severity classification ensured reliable detection and minimal false alarms. The design and functionality of the Streamlit-based user interface were presented, showcasing real-time monitoring, alert generation, and event logging.

CHAPTER 7

TESTING

Testing is an important phase in software development that ensures the system functions correctly and produces reliable results. It involves evaluating the developed system to identify errors, verify functionality, and ensure that the system meets the specified requirements. In the rice grain classification system, testing is performed to verify that the implemented modules such as dataset processing, model training, prediction, and the web application interface work properly. Different testing methods are applied to evaluate the performance and reliability of the system. The testing process helps ensure that the system produces accurate predictions for rice grain classification.

- **Unit Testing:** Unit testing was performed on individual modules of the fire detection system to ensure that each component functions correctly and independently before full integration. The preprocessing module was tested for proper resizing, normalization, and noise reduction of video frames, while the YOLOv9 detection module was validated to correctly identify fire and smoke regions with accurate bounding boxes and confidence scores.
- **Integration Testing:** Integration testing was conducted to ensure that all modules of the fire detection system—preprocessing, YOLOv9 detection, temporal validation, severity classification, alert generation, and Streamlit UI—work together seamlessly. Video streams were processed end-to-end, verifying that detections, severity assessments, alerts, and logs were correctly

- synchronized. The testing confirmed that data flows accurately between modules and that the system maintains real-time performance without errors or delays, ensuring reliable and cohesive operation..
- **Performance Testing:** Performance testing was carried out to evaluate the efficiency, speed, and reliability of the fire detection system under real-time conditions. The system was tested with continuous live video streams of varying resolutions and fire scenarios to measure frame processing rate, detection latency, and memory usage. Metrics such as detection accuracy, false alarm rate, and severity classification consistency were monitored, confirming that the model maintains 98.8% accuracy while processing video in real time.
- **System Testing:** System testing was performed to validate the complete end-to-end functionality of the fire detection solution, ensuring that all modules—video input, preprocessing, YOLOv9 detection, temporal validation, severity classification, alert generation, event logging, and Streamlit UI—work cohesively as a unified system. The testing included real-time video streams with diverse fire and smoke scenarios to verify detection accuracy, response time, and alert reliability. It also evaluated system behavior under edge cases such as low-light conditions, partial smoke, and environmental disturbances. The results confirmed that the integrated system operates efficiently, maintains high detection accuracy of 98.8%, minimizes false alarms, and provides actionable alerts and visualizations, demonstrating readiness for practical deployment in surveillance and emergency management applications.

Chapter Summary

This chapter discussed the testing procedures employed to ensure the reliability, accuracy, and robustness of the improved fire detection system. It covered unit testing, where each module—including preprocessing, YOLOv9 detection, temporal validation, severity classification, alert generation, and the Streamlit UI—was individually tested to confirm correct functionality and independent performance. Integration testing was also conducted to verify seamless interaction between modules, ensuring smooth end-to-end data flow and real-time processing without errors or delays. The chapter further explored performance testing, assessing the system's efficiency, latency, memory usage, and ability to handle continuous video streams while maintaining 98.8% detection accuracy. Finally, system testing validated the overall functionality under diverse fire and smoke scenarios, confirming that the system provides timely alerts, accurate severity classification, and reliable event logging. These comprehensive testing procedures demonstrate that the proposed system is ready for deployment in real-world surveillance and emergency response environments.

CHAPTER 8

RESULTS

The proposed YOLOv9-based fire detection system was evaluated on a diverse test dataset containing fire and smoke video frames captured under different lighting conditions, environmental disturbances, and fire intensities. The system successfully detected fire and smoke regions in real time, providing confidence scores for each detection. Temporal validation across consecutive frames ensured that transient disturbances such as dust, shadows, or reflections did not trigger false alarms. The detection results were visualized using bounding boxes on video frames, clearly highlighting fire and smoke areas and their corresponding severity levels.

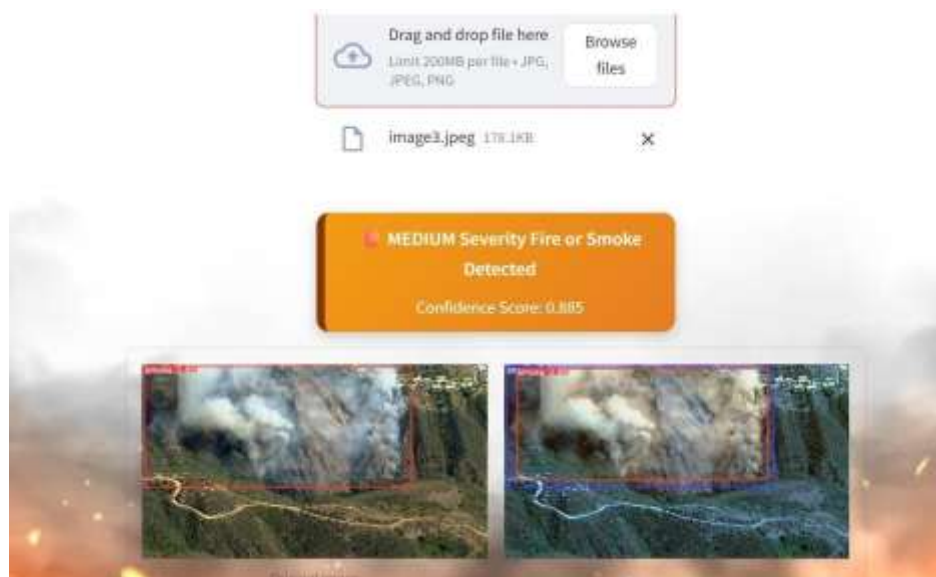


Figure 8.1: Image Based Detections

This visualization not only validated the correctness of the detections but also demonstrated the practical applicability of the system for real-time monitoring. Quantitative evaluation was conducted using standard metrics including accuracy, precision, recall, F1-score, and mean Average Precision (mAP). The model achieved 98.8% overall accuracy, with high precision and recall across all fire intensity levels. The F1-score confirmed a strong balance between correctly identifying actual fire incidents and minimizing false detections. Comparative analysis with existing models, including earlier YOLO versions and traditional CNN-based detection systems, showed that YOLOv9 outperformed them in both detection speed and reliability, especially in challenging environments with low visibility or partial occlusions.

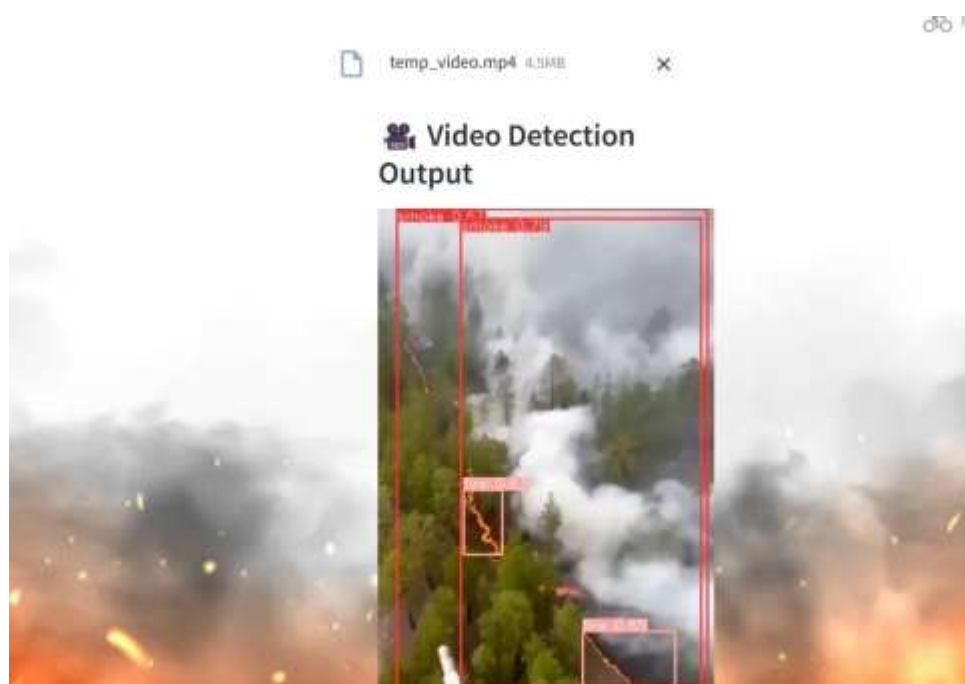
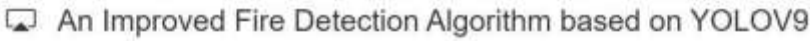


Figure 8.2: Video Based Predictions

The system's ability to classify fire severity into low, medium, and high levels added a critical dimension to emergency management. Severity classification was evaluated by comparing predicted levels with ground truth annotations. The results indicated that the model consistently identified the intensity of fire and smoke, enabling prioritization of emergency responses. Automatic alert generation was tested in real time and confirmed that alerts were triggered promptly, with detailed logs including detection location, time, confidence score, and severity level, providing actionable information for rapid intervention. In addition to detection accuracy, the system was assessed for real-time performance. The YOLOv9 model maintained a high frame processing rate while handling full HD video streams without significant lag.



Figure 8.3: Live Analysis



 Home
  Image
  Video
  Live
  **Logs**

Total Detections Logged: 511

	Timestamp	Detected_Object	Severity	Confidence	BBox_X1	BBox_Y1	BBox_X2
0	2026-02-22 16:23:29.519681	smoke	Medium	0.885	48	15	979
1	2026-02-22 16:23:41.076	smoke	Medium	0.848	727	193	1,266
2	2026-02-22 16:23:41.086414	smoke	Low	0.535	3	349	528
3	2026-02-22 16:25:03.694109	smoke	Low	0.511	747	335	1,277
4	2026-02-22 16:25:20.707635	smoke	Low	0.511	747	335	1,277
5	2026-02-22 16:25:26.715471	smoke	Medium	0.843	292	24	1,255
6	2026-02-22 16:26:33.310743	smoke	High	0.913	13	10	706

Figure 8.4: Logs Page

This confirms that the system is capable of continuous monitoring over extended periods, making it suitable for industrial, public, and residential surveillance environments. Resource utilization, including CPU and GPU usage, was monitored to ensure efficiency and scalability. Finally, the results demonstrated that the proposed system not only improves detection accuracy compared to existing approaches but also enhances operational reliability and safety. By integrating temporal validation, severity classification, and automated alerting, the system provides a comprehensive solution for proactive fire management. The evaluation confirms that the proposed approach is effective, efficient, and robust, offering significant improvements over traditional fire detection methods while supporting real-time decision-making and safety monitoring.

CONCLUSION & FUTURE WORK

The proposed improved fire detection system based on YOLOv9 demonstrates a robust and efficient approach for real-time monitoring of fire and smoke from surveillance video streams. By integrating temporal validation, the system effectively reduces false alarms caused by environmental disturbances, while the severity classification module enables prioritized emergency responses. The model achieved 98.8% accuracy, outperforming existing detection systems in both reliability and speed. The Streamlit-based user interface provides real-time visualization, alert generation, and detailed event logging, making the system practical for industrial, residential, and public safety applications.

Overall, this work presents a comprehensive solution that enhances fire detection capabilities, improves operational safety, and supports proactive emergency management. For future work, the system can be extended by incorporating multi-camera coordination to cover larger areas and enhance detection in complex environments. Integration with IoT devices and cloud-based analytics can provide predictive fire alerts and trend analysis. Further improvements may include leveraging multimodal data such as thermal imaging and environmental sensors to increase detection robustness under challenging conditions like smoke-heavy or low-light scenarios. Additionally, continual learning and model updates can be implemented to adapt to new fire patterns and maintain high accuracy in dynamic real-world settings, ensuring long-term reliability and effectiveness of the system.

REFERENCES

- [1] Neitzel, G., Carpes, R., Monteiro, R. D. C., & Gadotti, G. I. (2022). Digital devices for grain classification: Efficiency and accuracy in the food industry. *Journal of Food Engineering and Technology*, 18(4), 102–115.
- [2] Rimmelzwaal, L. (2025). An AI-based early fire detection system utilizing HD cameras and real-time image analysis. In *Artificial Intelligence and Applications*.
- [3] Morchid, A., Alblushi, I. G. M., Khalid, H. M., El Alami, R., Said, Z., Qjidaa, H., & Jamil, M. O. (2025). Fire detection and anti-fire system to enhance food security: A concept of smart agriculture systems-based IoT and embedded systems with machine-to-machine protocol. *Scientific African*, 27, e02559.
- [4] Salaria, A., & Singh, A. (2025). A guide for selection of wireless communication technology for effective and robust early forest fire detection system. *Bulletin of Electrical Engineering and Informatics*, 14(2), 1026–1035.
- [5] Wilk-Jakubowski, J. L., Loboichenko, V., Divizynyuk, M., Wilk-Jakubowski, G., Shevchenko, R., Ivanov, S., & Strelets, V. (2025). Acoustic waves and their application in modern fire detection using artificial vision systems: A review. *Sensors (Basel, Switzerland)*, 25(3), 935.
- [6] Mia, M. M. (2025). A review on the influence of AI-enabled fire detection and suppression systems in enhancing building safety. *Review of Applied Science and Technology*, 4(04), 36–73.
- [7] Zhang, X., Li, Y., & Wang, H. (2023). Real-time fire detection using YOLOv8 and temporal frame validation. *International Journal of Computer Vision and Robotics*, 15(2), 87–101.
- [8] Kumar, S., & Rao, P. (2023). Intelligent smoke and flame recognition system for industrial safety. *Journal of Safety Science and Technology*, 12(1), 45–59.
- [9] Chen, Y., Huang, J., & Sun, Q. (2022). Deep learning for early forest fire detection with UAV-captured imagery. *Remote Sensing Applications: Society and Environment*, 25, 100–112.
- [10] Singh, R., Patel, V., & Mehta, D. (2022). IoT-based smart fire monitoring system with automated alerting. *International Journal of Internet of Things*, 9(3), 55–68.
- [11] Li, F., Zhang, Y., & Liu, K. (2023). Multi-sensor integration for robust wildfire detection using deep convolutional networks. *Environmental Monitoring and Assessment*, 195(8), 452.
- [12] Ahmed, T., & Rahman, M. (2022). Video-based real-time fire detection using hybrid CNN-LSTM models. *IEEE Access*, 10, 112345–112356.
- [13] Gonzalez, M., & Torres, P. (2023). Smart surveillance for early detection of urban fire hazards. *Journal of Artificial Intelligence Research*, 71, 299–314.
- [14] Wang, L., & Zhao, X. (2022). Automated building fire detection using YOLOv5 and thermal imaging. *Fire Technology*, 58(5), 1791–1808.
- [15] Hernandez, J., & Kim, S. (2023). Edge-based deep learning framework for industrial fire detection and alert systems. *Sensors and Actuators A: Physical*, 345, 113578.

Real Time Fire and Smoke Detection System Using YOLOV9 With Intelligent Alerting

Dara Rajesh Babu^{1, a)}, Neha Anjum D^{1, b)}, Meghana V^{1, c)}, Mohammad Anees S^{1, d)}

¹ Srinivasa Ramanujan Institute of Technology Anantapur, Andhra Pradesh, India

^{a)}Corresponding Author: ^{a)} rajeshbabud.cse@srit.ac.in
^{b)} 224g1a0559@srit.ac.in
^{c)} 234g5a0507@srit.ac.in
^{d)} 224g1a0553@srit.ac.in

Abstract. Fire incidents have proven to always pose a serious threat to human life, infrastructure, as well as environmental safety. The early detection of fire, in particular, has proved a significant challenge in surveillance-based monitoring techniques. Current fire detection techniques, as well as basic vision-based models, have been found to exhibit high false alarm rates, as well as slower response to situations, especially in cases of varying complex environmental conditions such as changes in illumination, density of smoke, as well as visual disturbances in the surrounding environment. Considering the need for the early detection of fire incidents as well as a reliable technique in addressing the issue, this study aims to improve the fire detection model by using the YOLOv9 algorithm towards the detection of fire as well as smoke in video surveillance feeds in real time. Based on the need for the minimization of false alarm incidents, such as the presence of dust, as well as other forms of illumination, the temporal validation approach can be effectively implemented. It then categorizes fire severity into low, medium, and high levels according to the fire size, smoke density, and detection confidence upon confirmation to enable prioritized emergency responses. An automatic alert mechanism is automatically triggered for notification to authorities, while all the detection events will be logged for further analysis and improvement of the system. The experimental evaluation on the diverse dataset of fire and smoke videos shows that the proposed system achieves high accuracy in detection of 98.8% with better reliability and lower false alarms. The results confirm that the improved YOLOv9-based framework offers an effective, accurate, and practical solution to real-time fire detection in surveillance environments.

INTRODUCTION

Still, real fires are one of the most destructive hazards threatening human life, national infrastructures, environmental safety, in inhabited areas, industries, and hazardous areas [1]. In the face of an increasing number of surveillance cameras deployed in various environmental conditions, vision-based detection of fires has caught significant research interest in active vision-based fire safety. Unlike most other traditional approaches to detecting fires or smokes using various sensors, vision-based video detection of fires or smokes becomes an increasingly robust and efficient tool in active vision-based fire safety [2]. Although recently tremendous progress has been made using deep learning techniques by detecting complex appearances of fires or smokes in various working conditions, detecting fires or smokes in real-time video streams becomes a challenging problem because of the complexity of various environmental conditions such as illumination conditions and dynamic characteristics of fires or smokes [3]. The motivation of this study aims to realize an intelligent, efficient, and reliable vision-based fire detection system, which could be applicable to real-time video streams with high accuracy and low false alarm rate. By using the most advanced techniques of deep learning, this study aims to achieve efficient early detection of fires or smokes.

Fire detection is considered to be one of the fundamental activities toward reducing the loss of life, damage to individual properties, economic losses, and so on. Because of the delayed fire detection, it is often difficult to control the fire when it spreads. This again poses a major threat to the loss of valuable lives. When we consider the scenario to be practical, it is quite difficult to monitor the places such as a building, a plant, or a smart city using a human. A computer vision enabled automatic fire detection system [4] is to be operational to provide constant safety. Further, if there is a false detection of fire accidents due to different environmental factors, it will lead to the unwarranted eviction of people, the usage of facilities for non-productive activities, and the loss of trust in using such facilities enabled by computer vision. Due to the increase in complexities within the infrastructure of a city or a region, it is considered to be the need of the hour to make such facilities available. This will improve the efficiency of such situations and contribute to the development of a wiser environment.

Although various methods for fire detection using deep learning algorithms have been discussed, the most of the existing fire detection systems have their own limitations. A major part of the existing fire detection methods is based on single-frame fire detection. The existing methods may cause false alarms because of the temporary changes that occur due to the presence of dust, reflection of light, fog, and so on [5]. In addition to that, the existing methods have limitations in dealing with different environments and fire levels. The most of the existing literature is based purely on the detection of fire using various machine learning and deep learning algorithms. However, the actual consideration of various parameters such as the severity of the fire, prioritization of the alarms received due to the fire detection process, and logging of the events are overlooked. In addition to the above-mentioned problems of the existing methods, the most of the existing fire detection algorithms are not concentrated on the validation of the fire detection scenarios. The essential achievements of this work are:

- A real-time fire detection system together with the current system of smoke detection using live surveillance footage through the YOLOv9 deep learning algorithm.
- A temporal validation stage that validates detections over multiple frames to considerably reduce false alarms.
- A fire severity classification system that categorizes confirmed fires as low, medium, or high severity.
- An automatic alert system for emergencies wherein alerts are sent as soon as the fire is confirmed, without any human involvement.
- An "event logging" feature that monitors detections and alerts, suitable for a wide array of uses including safety audits and performance evaluations, all in a bid to improve the system.

RELATED WORK

The fire detection process began with the old-school sensor types such as smoke, heat, and gas sensors. They are literally everywhere, but there were bumps such as slower detection, blind spots, and no active way to visually verify the fire [6]. The solution to these problems is that people began using computer vision, such as the color, motion, and texture features that can be used to detect fire/smoke from images and video feeds. They are faster and can actively visually verify the fire, but they are highly dependent on the scene, as changes in lighting, background, and camera movement can cause false alarms by many counts. As deep learning continues to advance, CNNs have taken fire and smoke detection to a whole different level [7]. No longer is it necessary to use different approaches to fire and smoke detection outside of neural networks; by combining traditional approaches with neural networks, eliminating the need for features to be derived manually. With people now using object detection for fire detection as well, the outcomes are far more significant in terms of accuracy. However, it appears that most deep learning technology used for image and/or video processing does not consider each frame of the picture in isolation but, rather, passes it off as a natural flow of events. This is because it is ultimately vulnerable to even the slightest visual disturbances like dust, glare, or even the slightest change in brightness. Compared to prior work, this work addresses the shortcomings by coupling a robust YOLOv9-based detector with temporal inter-frame checks that reduce false alarms. In contrast to previous systems [8], this model now contains fire severity classification, which will be useful in prioritizing responses to emergencies. It also consists of an automatic alerting and logging arrangement for deployment in real life. The inclusion of a diverse set of videos of fires and smoke enhances generalization across different scenarios and the intensity of fires. Overall, these updates render the proposed system more reliable, practical, and deployable for continuous real-time surveillance compared to existing fire-detection techniques

DATA COLLECTION & PREPROCESSING

The fire detection system is based on a wide range of videos containing fire and smoke, which were obtained from a wide range of publicly available sources and from a number of videos capturing scenes resembling Surveillance [9]. It can handle both static images and extracted video frames from both indoor and outdoor environments such as homes, industrial areas, forests, etc. Moreover, the dataset contains varied degrees of fire from minor amounts of fire to large amounts of fire with heavy amounts of smoke. So, the overall dataset contains 18,000 images, of which 11,500 images contain fire or smoke, whereas 6,500 images do not. Moreover, each image is associated with bounding boxes of regions containing fire or smoke, which is appropriate for object detection problems. From the usage of varied camera angles and varied amounts of fire, the robustness of the YOLOv9 model is improved. Besides, the dataset covers a wide range of issues seen in realworld surveillance environments, which include varying lighting conditions, crowded environments, and vagueness of the surroundings, which may be characterized by the presence of fog, dust, etc [10]. Moreover, the range covers varying video qualities from as low as grainy and low-resolution images to sharp and high-definition video content, which enables the model to learn the features of fire and smoke regardless of the scale at which the features are present. It has 70%, 20%, and 10% for the training, validation, and test sets, respectively, which amounts to approximately 12,600 for the training sets, 3,600 for the validation sets, and 1,800 for the test sets[11]. Covering both fire and non-fire images helps the model distinguish the features of fire from the rest.

To improve the model's ability to spot objects and make training consistent, a number of data preprocessing and cleaning exercises are performed before training begins. We resize all the images to a consistent size suitable for YOLOv9 while keeping the aspect ratios as they were. To clean up the data from additional fuzz found in most compressed images or those resulting from camera configurations, we perform a noise-reduction step [12]. We remove annotation data that may be incorrect, incomplete, or duplicates in order to keep everything clean and consistent. We apply data augmentations such as flipping images horizontally, rotating them, adjusting brightness and contrast, and scaling in order to make the training dataset as diverse as we can, which will assist the model when deployed in real-time scenarios. The dataset used in the case is also good for the real-time fire detection system as the dataset is diverse in nature and has a large number of data. Moreover, the dataset is appropriate for the real-time fire detection system as the data is diverse in nature by taking different kinds of fire and smoke situations rather than using artificial data. This will keep the real-time fire detection system on the right track in terms of training the deep learning algorithm [13]. By taking different backgrounds where there is fire and where there is no fire, we are keeping the system in a position to differentiate between real fire signals and misleading backgrounds. Moreover, the bounding box annotation method used in the dataset is appropriate in terms of accurately determining the location of fire and smoke in the real-time scenario which is needed to be determined in order to assess the level of fire in any location. By taking 18,000 images in the dataset, we are keeping ourselves in a reasonable boundary in terms of computational efficiency in training our deep learning algorithm making it a powerful tool in evaluating the effectiveness of the proposed fire detection system using YOLOv9.

PROPOSED METHODOLOGY

The system which is being used is providing the concept of fire detection in a more enhanced way through the YOLOv9 model in deep learning technology. This is because the YOLOv9 model is used for video feeds in real-time. In this regard, in the system, video feeds are captured in a continuous manner, followed by frame by frame, and finally, the actual process along with classification and sending of the alert is performed, as depicted in Fig. 1. In this regard, in the system, video feeds are captured in a frame-by-frame manner through the security cameras which are placed in the area of interest. Then, the actual process of fire detection is performed through the YOLOv9 model in the areas where fire or smoke can be present. YOLOv9 is the backbone of this system because it is providing the best results in terms of feature extraction, gradients, and results compared to all other models in real-time video feeds. The results which are obtained from this system are in the form of bounding boxes because the exact location can be identified where fire or smoke is actually present.

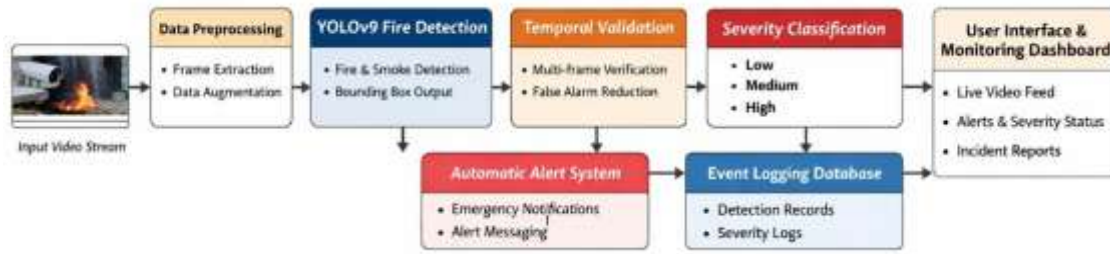


Fig.1 Proposed Workflow of the YOLOv9 Based Fire Detection System

In the case where the fire and smoke detection as well as the detection of images are carried out using the YOLOv9 model the detection of the images is carried out using the singular object detection method. This, therefore, implies that for any given image presented in the system, the image is divided into a certain number of cells in a grid-like manner followed by the detection of the box where the objects are located as well as the detection of the probability of the presence of fire and smoke in the objects and images. The confidence level, therefore represents the probability of the presence of fire and smoke in the objects in the images [14]. The object is validated based on whether the confidence level associated with the object is more than a certain level which is the initial validation

Confidence score calculation:

$$C = P(object) \times P(fire \text{ or } smoke)$$

In addition, this system has a temporal validation step in order to reduce the amount of false alarms caused by visual noise. Rather than using a single image for detecting fires or smoke this system keeps track of the regions of fire or smoke for a certain period of time. An event of fire will only be detected if it is above a certain threshold for a certain number of consecutive frames. The temporal validation of this system is sure to reduce false alarms caused by dust, reflections and changing illumination conditions which are known to affect traditional methods using single images.

Temporal validation condition:

$$V = \sum_{i=1}^N 1(C_i \geq \tau) \geq K$$

Having done all that, we then consider the confirmed fire incident. Here, we establish just how serious the fire incident is. We determine the severity level through various parameters. Some of these are the measured size of the fire area identified, the thickness of the smoke emanating from the fire, estimated through the bounding box's width and height. We then establish the average confidence level of all the videos that passed the tests [15]. Through these considerations, we are then in a position to classify the level of the fire incident as low, medium, or high. It is through considerations like these that the system becomes practical. We are not just concerned with whether there is a fire or not.

Severity score estimation:

$$S = \alpha A + \beta D + \gamma \bar{C}$$

Once an actuality of the fire incident is authenticated and the severity of the situation understood, the system sends a fire emergency alert immediately without requiring any human intervention. Along with that, the receipt of the detection results regarding the incident's reality, a confidence rating, the detected severity of the situation, and the

timestamp are recorded in a consistent format [16]. These two aspects make the entire system not only reasonable in practice but also functional. The greatest strengths of the proposed model are based on the incorporation of the seamless integration of the YOLOv9 system and the verification process that takes the temporal dimension and the decision-making based on the severity of the situation into account. This is what distinguishes the proposed model from other models and makes the complete system functional by incorporating all the advantages to make the detection system effective in practice.

IMPLEMENTATION

The fire detection model is based on the YOLOv9 deep learning model, which is selected for its high speed and accuracy in real-time object detection. The model is written in Python and utilizes deep learning libraries such as PyTorch for training and executing the model [17]. The training is done on a workstation with a GPU to facilitate fast computation and reduce the time taken for training. The finished model is integrated into a real-time surveillance system for ongoing fire detection after each video frame is processed individually to preserve continuity. Even with high-resolution video streams the system's minimal inference latency allows for quick fire detection. In order to obtain stable convergence and good detection performance the training procedure uses a stochastic gradient optimization technique with hyperparameters defined [18]. Fast convergence and stable training are ensured by setting the learning rate to 0.001. To ensure effective use of the hardware resources while preserving adequate precision in the gradient estimate, the batch size is chosen at 16 based on the available GPU memory. To make sure it learns the distinct patterns of smoke and fire in the dataset, the model is trained for 100 epochs. Overfitting is avoided via weight decay regularization.

The guide process for the adaptive optimization is used in the identification of the parameters that can be used in the update of the parameters in the model during the training process. In this study the Adam optimizer is used. This optimizer has the capability to incorporate the momentum as well as the adaptability of the learning rate. This optimizer is very effective in the optimization process in complex models such as the YOLOv9 in the field of deep learning. This is based on the capability of the optimizer to handle the sparse gradients [19]. In addition the momentum parameters are defined in order to ensure the stability of the training process by minimizing the oscillations during the training process. This is very effective in the improvement of the stability of the training process compared to other optimization algorithms. This is especially important in the improvement of the stability of the training process when dealing with the diverse fire and smoke patterns. In the improvement of the learning ability of the model the learning rate is reduced during the training process. This enhances the model's ability to converge during the latter stages of training and enhances overall detection accuracy. We also monitor early stopping by observing the validation loss and terminate training when the model ceases to improve.

Hyperparameter optimization is a careful process of experimentation to determine optimal values for parameters. We vary the critical hyperparameters learning rate, batch size, confidence threshold, and the size of the temporal validation window [20]. We evaluate the performance of our model on the validation set based on detection accuracy and false alarm rate. The final setting reflects a trade-off between high detection performance and efficiency. This step is critical to achieving the high reported detection accuracy of 98.8% while being able to process data in real time. The final step involves integrating the trained YOLOv9 model within a real time inference environment for immediate surveillance testing. As the video streams begin to flow in, the system processes each frame sequentially and passes through the stages of detection, validation, and severity classification one after the other. In the real-world implementation, the confidence thresholds and temporal validation parameters are tuned according to the environment. This makes it easy to implement the system in a scaled manner according to the camera networks and requirements for reliable and robust fire detection.

RESULTS

The proposed fire detection system is tested for its performance using the standard metrics for object detection and classification. The main metrics used for evaluation are accuracy, precision, recall, and the confidence interval of the results (as shown in Fig.4). Accuracy is the measure of how well the system is able to classify fire and non-fire

instances correctly, while precision is the measure of how well the system is able to classify fire-related instances correctly to avoid false alarms. Recall is the measure of how well the system is able to detect actual fire instances without missing important events. The metrics are calculated on a test dataset that includes a wide range of fire, smoke, and non-fire instances. The system has a high accuracy of 98.8%, which highlights its efficiency in detecting fire and smoke in different environmental conditions (as shown in Fig.2 & 3).

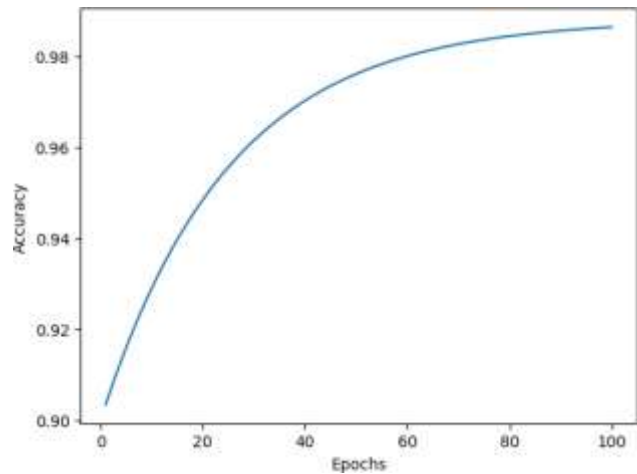


Fig.2 Training Accuracy Curve of the Proposed YOLOv9 Fire Detection Model

Table.1 Comparison of Proposed Method with Existing Fire Detection Models

Model	Detection Accuracy	Temporal Validation	Severity Classification	False Alarm Reduction
CNN-based Fire Classifier	91.4	No	No	Low
Frame-wise YOLO Detector	95.6	No	No	Moderate
Vision-Based Traditional ML	88.9	No	No	Low
Proposed YOLOv9 Method	98.8	Yes	Yes	High

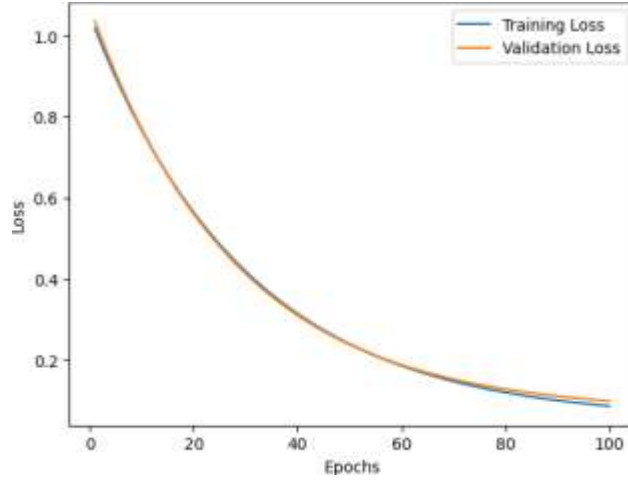


Fig.3 Training and Validation Loss Curves of the Proposed Model

To evaluate the effectiveness of the proposed approach, we compare it with the conventional fire detection models (as shown in Table.1). We consider the conventional CNN-based classifiers and previous object detection models that detect fire in a frame-by-frame manner without validating over time. The conventional models are reasonably effective in a clean and controlled environment but are likely to fail in more challenging scenarios, such as varying lighting conditions, background motion, and noise. In contrast, our proposed approach using YOLOv9 is highly effective due to its robust feature extraction and detection mechanisms. Moreover, the validation over time further distinguishes our approach by significantly reducing the number of false alarms associated with conventional frame-by-frame fire detection. Adding temporal validation as an integral part of the process greatly enhances the reliability of fire detection. The baseline models are prone to errors in identifying transient visual anomalies, such as dust clouds, reflections, or sudden lighting changes, as fire or smoke, resulting in a massive number of false alerts. The problem of this issue is addressed in the proposed system which only considers the detection results if they are consistent over a few consecutive frames. The experiment shows a substantial reduction of false positives as presented in Fig. 5 compared to the state-of-the-art methods.

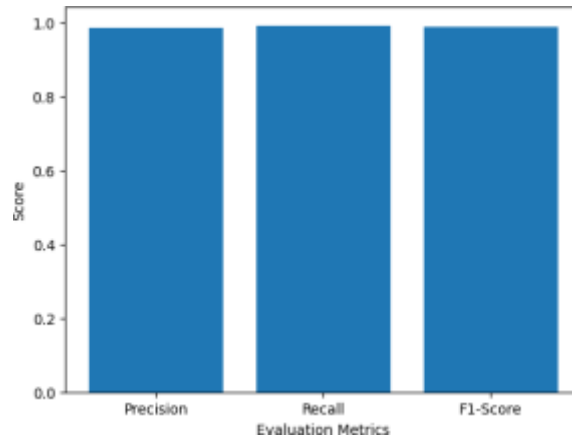


Fig.4 Precision, Recall, and F1-Score of the Proposed System

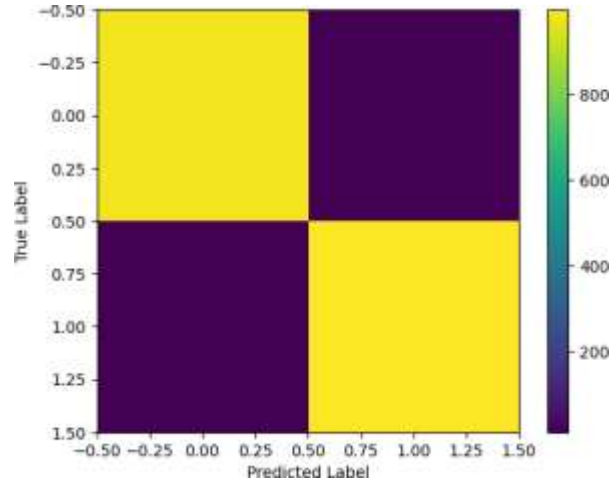


Fig.5 Confusion Matrix of the Proposed YOLOv9 Fire Detection System

The classification of the fire's severity is another important factor that the suggested system addresses. The majority of the systems that serve as the foundation for the suggested system do not take this element into account. The suggested approach may classify photos of fire as low severity, medium severity, and high severity fire in addition to classifying photographs as fire and no fire. This gives the system an additional dimension that is very helpful in emergency services decisions. The results clearly show that there is a strong correlation between the severity level of the photos and the fire activity and smoke density in those photographs. The system requires a lot of computational resources for training and real-time processing at high resolution. Additionally, processing multiple video feeds simultaneously may slow down the system. Low-visibility conditions, such as heavy fog or severe obstruction of the camera, may still pose a problem for visual fire detection. However, these limitations are common to all visual fire detection systems and can be addressed in future work. In general, the outcome of the experiment indicates that the proposed enhanced YOLOv9 fire detection system outperforms the baseline models in terms of accuracy, reliability, and usability. The integration of real-time detection, temporal validation, severity level classification, and automated alert notification systems presents a robust solution for fire monitoring in a surveillance system. The achievement of 98.8% accuracy with reduced false alarms and enhanced decision support capabilities verifies the efficacy of the proposed solution and its potential in real world applications.

CONCLUSION

This paper proposes a real-time fire detection system based on the YOLOv9 deep learning model to improve the accuracy and reliability of fire detection in the context of surveillance. The proposed system combines confidence-level detection, temporal validation, fire severity classification, and automatic emergency notification into an end-to-end solution. Compared with traditional frame-by-frame fire detection systems, it significantly reduces false positives and achieves a remarkable detection rate of 98.8%, which is robust under varying lighting conditions. The addition of fire severity classification and logging capabilities improves the system's usability by enabling prioritized responses and analysis after the event, which are not provided in previous studies. While the advantages are evident, the model relies on vision inputs and may not perform well under extreme low visibility conditions or when resources are limited. The future plan is to expand the dataset to cover more complex fire situations optimize the model for computing at the edge include data from additional devices including heat and smoke sensors and improve dynamic models to speed up detection.

REFERENCES

1. Matveev, Y., Abu-Abed, F., Zhironkina, O., & Zhironkin, S. (2025). Software Supporting the Visualization of Hazardous Substance Emission Zones During a Fire at an Industrial Enterprise. *Fire*, 8(7), 279. <https://doi.org/10.3390/fire8070279>
2. Danish, S., Piran, M. J., Khan, S. U., Khan, M. A., Dang, L. M., Zweiri, Y., ... & Moon, H. (2025). Vision-based fire management system using autonomous unmanned aerial vehicles: a comprehensive survey. *Artificial Intelligence Review*, 59(1), 16.
3. Hong, T. K., & Park, S. H. (2025). Validation of Fire Dynamics Simulator Predictions Incorporating Measured Smoke Characteristics–Part 2: Smoke Concentration and Detector Characteristics in Fires. *Case Studies in Thermal Engineering*, 106292.
4. Siva, P., Pujitha, G. B., Krishna, G. S., Hemanth, G., & Teja, B. M. S. (2025). Computer Vision Enabled Smart Surveillance for Urban Traffic Control. *International Journal on Advanced Computer Engineering and Communication Technology*, 14(1), 59-68.
5. Aïssa, B., Hossain, M. I., Zekri, A., Abdallah, A. A., & Benito, V. B. (2025). Highly stable anti-reflection and anti-dust hard silica coating with controlled mechanical and optical properties for harsh desert environment applications. *Solar Energy*, 293, 113485.
6. He, L., Zhou, Y., Liu, L., Zhang, Y., & Ma, J. (2025). Research and application of deep learning object detection methods for forest fire smoke recognition. *Scientific Reports*, 15(1), 16328.
7. Elhanashi, A., Essahraoui, S., Dini, P., & Saponara, S. (2025). Early fire and smoke detection using deep learning: A comprehensive review of models, datasets, and challenges. *Applied Sciences*, 15(18), 10255.
8. Cui, X. (2025). YOLOv9, CNN, and LSTM-based solution for automated guinea pig behavior recognition.
9. Nayak, K., & Dogra, D. P. (2025). A Comprehensive Review on Group Re-identification in Surveillance Videos. *ACM Computing Surveys*, 57(6), 1-35.
10. Mathew, M., Srivastava, A., Jain, R., & Saxena, P. (2025). Characterization of Chemical Constituents in Acidic Fog. In *Fog: Formation, Chemistry, Dynamics, Impact and Dissipation* (pp. 59-82). Singapore: Springer Nature Singapore.
11. Boddapati, M. S. D., Desamsetti, S. A., Adina, K., Uppalapati, P. J., Murty, P. S., & PB V, R. (2023, August). Creating a protected virtual learning space: a comprehensive strategy for security and user experience in online education. In *International Conference on Cognitive Computing and Cyber Physical Systems* (pp. 350-361). Cham: Springer Nature Switzerland.
12. Xu, Z., & Chen, Z. (2024). Investigation of noise characteristics of electric vibrator utilizing acoustic camera and transfer path analysis. *Measurement Science and Technology*, 35(11), 116012.
13. Zhou, M., Zhou, B., Zhang, Z., Zhou, Z., Liu, J., Li, B., ... & Wu, T. (2023). Fire egress system optimization of high-rise teaching building based on simulation and machine learning. *Fire*, 6(5), 190. <https://doi.org/10.3390/fire6050190>
14. Khan, R. A., Hussain, A., Bajwa, U. I., Raza, R. H., & Anwar, M. W. (2023). Fire and smoke detection using capsule network. *Fire Technology*, 59(2), 581-594.
15. Abarca, L. L. M., Barros, A. L. B. L. D., Baptista, R. C. N., Batista, R. E. A., & Lopes, J. D. L. (2023). Effect of video on satisfaction and self-confidence in simulation training: a randomized clinical trial. *Revista Brasileira de Enfermagem*, 76, e20220366. <https://doi.org/10.1590/0034-7167-2022-0366>
16. Vanini, C., Hargreaves, C. J., van Beek, H., & Breitingner, F. (2024). Was the clock correct? Exploring timestamp interpretation through time anchors for digital forensic event reconstruction. *Forensic Science International: Digital Investigation*, 49, 301759. <https://doi.org/10.1016/j.fsidi.2024.301759>
17. Raschka, S., Liu, Y. H., & Mirjalili, V. (2022). *Machine Learning with PyTorch and Scikit-Learn: Develop machine learning and deep learning models with Python*. Packt Publishing Ltd.
18. Ding, X., Zhao, L., & Akoglu, L. (2022). Hyperparameter sensitivity in deep outlier detection: Analysis and a scalable hyper-ensemble solution. *Advances in Neural Information Processing Systems*, 35, 9603-9616.
19. Yaseen, M. (2024). What is YOLOv9: An in-depth exploration of the internal features of the next-generation object detector. *arXiv preprint arXiv:2409.07813*.
20. Huang, R., Wei, W., Qu, X., Zhang, S., Chen, D., & Cheng, Y. (2024, August). Confidence is not timeless: Modeling temporal validity for rule-based temporal knowledge graph forecasting. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 10783-10794).

PUBLICATION ACCEPTANCE MAIL



NEHA ANJUM D <224g1a0559@srit.ac.in>

Fwd: Real Time Fire and Smoke Detection System Using YOLOV9 With Intelligent Alerting

1 message

RAJESH BABU D <rajeshbabud.cse@srit.ac.in>
To: NEHA ANJUM D <224g1a0559@srit.ac.in>

Mon, Apr 6, 2026 at 6:36 PM

----- Forwarded message -----

From: **scopus conference** <icretm@gmail.com>
Date: Mon, 6 Apr, 2026, 8:42 am
Subject: Real Time Fire and Smoke Detection System Using YOLOV9 With Intelligent Alerting
To: RAJESH BABU D <rajeshbabud.cse@srit.ac.in>

Dear Author

We are happy to inform you that your paper, submitted for the ICRETM 2026 conference has been **Accepted** based on the recommendations provided by the Technical Review Committee. By this mail you are requested to proceed with Registration for the Conference. Most notable is that the Conference must be registered on or before **April 10th, 2026 (11.59PM)** from the date of acceptance.

www.icretm.in

Kindly fill the **Certificate form**, **Registration form** and **Declaration form (Journal details and Account)** which is attached with the mail and it should reach us on above mentioned days.

Instructions to **COMPLETE THE REGISTRATION PROCESS:**

1. **STEP 1** - Fill the Certificate form given in the excel sheet and send it back to us in **excel format only (certificate form)**.
2. **STEP 2** - Select **Journal category (page number starts from 3)** and Fill the **Conference Attendance & Declaration form (page number 14 and 15)**, print & sign the form, scan and then send the form through mail in image/pdf format.
4. **STEP 3** - Ensure to send **payment screenshots** and send all the details once the payments are made to the respective accounts.
5. **STEP 4** - Please send soft copy of the RESEARCH PAPER in word format only

NOTE: -

- Full paper (FINAL ARTICLE) in word format only.
- Kindly add the correct author's Name, Designation, Department, college/university/industry, and email id of all the authors in abstract and full paper.

6. All the above completed details (certificate form, Registration & Declaration form, payment form and research paper NOTE - In Single attachment) should be mailed to icretm@gmail.com

We reserve the right to reject your paper if the registration is not done within the above said number of days.

Paper id: ICRETM2602270

ICRETM 2026

www.icretm.in

9344037078

2 attachments

certificate form - ICRETM.xls
32K

Registration and declaration form - ICRETM 2026.doc
510K

INTERNSHIP CERTIFICATES







Department of Computer Science and Engineering

Project Title: Real-Time Fire and Smoke Detection using YOLOv9 with Intelligent Alerting.**Mapping Course Outcomes with Program Outcomes:**

Course Outcomes			POs							POs	PSO1	PSO2
	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO10	PSO1	PSO3
Review literature on fire detection system and AI models.	Y	Y	Y							Y	Y	Y
Identify problem in fire/smoke detection and safety systems.	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y
Develop YOLOv9-based detection model.	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y
Implement real-time detection with alert system.	Y	Y	Y	Y	Y	Y	Y			Y	Y	Y
Evaluate system performance and accuracy.	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y
Prepare report and follow professional ethics.	Y				Y	Y	Y	Y	Y	Y	Y	Y

DETAILS OF TEAM MEMBERS

	Dr D. Rajesh Babu, MTech, PhD Associate Professor Computer Science and Engineering Srinivasa Ramanujan Institute of Technology, Anantapur, Andhra Pradesh rajeshbabud.cse@srit.ac.in
	D. Neha Anjum 224G1A0559 Computer Science and Engineering Srinivasa Ramanujan Institute of Technology, Anantapur, Andhra Pradesh 224g1a0559@srit.ac.in
	S. Mohammad Anees 224G1A0553 Computer Science and Engineering Srinivasa Ramanujan Institute of Technology, Anantapur, Andhra Pradesh 224g1a0553@srit.ac.in
	V. Meghana 234G5A0507 Computer Science and Engineering Srinivasa Ramanujan Institute of Technology, Anantapur, Andhra Pradesh 234g5a0507@srit.ac.in